KAG-Thinker: Interactive Thinking and Deep Reasoning in LLMs via Knowledge-Augmented Generation

Knowledge Engine Team @Inclusion AI, Ant Group*

*See Contributions section (Sec. 7) for full author list.

In this paper, we introduce KAG-Thinker, which upgrade KAG to a multi-turn interactive thinking and deep reasoning framework powered by a dedicated parameter-light large language model (LLM). Our approach constructs a structured thinking process for solving complex problems, enhancing the the logical coherence and contextual consistency of the reasoning process in question-answering (Q&A) tasks on domain-specific knowledge bases (KBs) within LLMs. Following the **Logical Form** guided retrieval and reasoning technology route of KAG, this framework first decomposes complex questions into independently solvable sub-problems (which are also referred to as logical forms) through **breadth decomposition**. Each such logical form is represented in two equivalent forms—natural language and logical function—and subsequently classified as either a Knowledge Retrieval or Reasoning Analysis task. Dependencies and parameter passing between these tasks are explicitly modeled via logical function interfaces. In the solving process, the Retrieval function performs retrieval tasks. It retrieves one-hop structured and unstructured information of specified knowledge unit. While the Math and Deduce functions are used to perform reasoning analysis tasks. Secondly, it is worth noting that, in the Knowledge Retrieval sub-problem tasks, LLMs and external knowledge sources are regarded as equivalent KBs. We use the **knowledge boundary** module to determine the optimal source using self-regulatory mechanisms such as confidence calibration and reflective reasoning, and use the **depth solving** module to enhance the comprehensiveness of knowledge acquisition. Finally, instead of utilizing reinforcement learning, we employ supervised fine-tuning with multi-turn interactive thinking and deep reasoning to align the model with our structured reasoning paradigm, thereby avoiding excessive reflection. This is supported by a data evaluation framework and iterative corpus synthesis, which facilitate the generation of detailed reasoning trajectories. Experimental results show that our model outperforms state-of-the-art trained deep search models on seven benchmark datasets, achieving an average improvement of 4.1%. We further demonstrate its practical effectiveness through a medical Q&A system integrated with domain-specific knowledge. We train KAG-Med-Thinker model with 14b-parameter based on synthetic medical corpora using the same approach, respectively, and verify their applicability in specialized applications.

Code of KAG-Thinker: https://github.com/OpenSPG/KAG-Thinker
Code of KAG Framework: https://github.com/OpenSPG/KAG

1 Introduction

Recent advances in large language models (LLMs) OpenAI et al. (2024); Qwen et al. (2025) have shown impressive performance in Q&A tasks. However, their ability to solve multi-hop problems Wei et al. (2022), especially those that require external retrieval of up-to-date information Jin et al. (2025a), has certain limitations. Previous work primarily focused on how to use external retrievers, broadly falling into two categories: (1) decoupling retrieval from training, and (2) autonomously deciding whether to use the retriever during training. The first category primarily involves retrieval augmentation generation (RAG) with predefined workflow, the representative works include Lewis et al. (2020), Trivedi et al. (2023), and Li et al. (2025a). These models often use the question to retrieve relevant passages, which are then added to the LLM's context. This helps the LLM leverage external knowledge when answering questions. The second category mainly concerns to mimic the deeper and more deliberative reasoning as "slow thinking" Pan et al. (2025), a paradigm inspired by human cognitive processes, to determine the timing of retrieval dynamically. Exemplified by Jin et al. (2025b), Sun et al. (2025), and Wang et al. (2025c). These methods generate a specific token <search> that indicates an external search engine query is required. Once the retriever provides results, these are then fed back into the model as input, allowing it to resume content generation. Although these methods improved LLM utilization of retrievers, they do not truly provide a step-by-step approach for solving complex multi-hop problems. Their approaches are more concerned with how LLMs arrive at the correct answer, without ensuring the rigor, stability, or logical consistency of the underlying problem-solving process. Consequently, these methods are often unreliable in high-stakes or sensitive domains such as finance, medicine, and legal applications.

In order to achieve logical reasoning with more accurate judgments and reduced biases (Li et al., 2025b), recent work has attempted various approaches, such as the atomic reasoning actions decomposition by Wu et al. (2024) , hierarchical and structured reasoning by Yang et al. (2025) , and verify reasoning trajectories with symbolic solver by Guan et al. (2025) . Moreover, Sumers et al. (2024) propose a cognitive architectures for language agents with three key dimensions: information storage, action space and decision-making procedure. Benefit from these previous works and inspired by Slow Thinking - which solves complex problem with access to additional computational resources, full attention, and sophisticated logical reasoning Booch et al. (2021) - we propose an innovative structured thinking and reasoning model as shown in Figure 1. It enables deep thinking in LLM by simulating human cognitive mechanisms without reinforcement learning DeepSeek-AI et al. (2025); OpenAI (2024). We use logical form to decompose complex problems broadly and solve them in depth. Logical form improves the logic, rigor, and stability of the LLM reasoning process. When humans solve complex problems, they only perform external information retrieval on knowledge that they do not know. Similarly to this process, we do not need to perform external knowledge retrieval for all decomposed sub-problems. Therefore, we design an LLM's knowledge boundary determination module to reduce unnecessary external retrieval and mitigate noise from the retrieved content. If the LLM cannot confidently answer a certain sub-question using its own knowledge, we rely on external KBs. However, search results from external KB are often unreliable and noisy. To address this, we created a Focusing-and-Reasoning module to extract the core relevant information. To facilitate a clearer understanding, we will introduce these key innovations in detail in the following paragraphs.

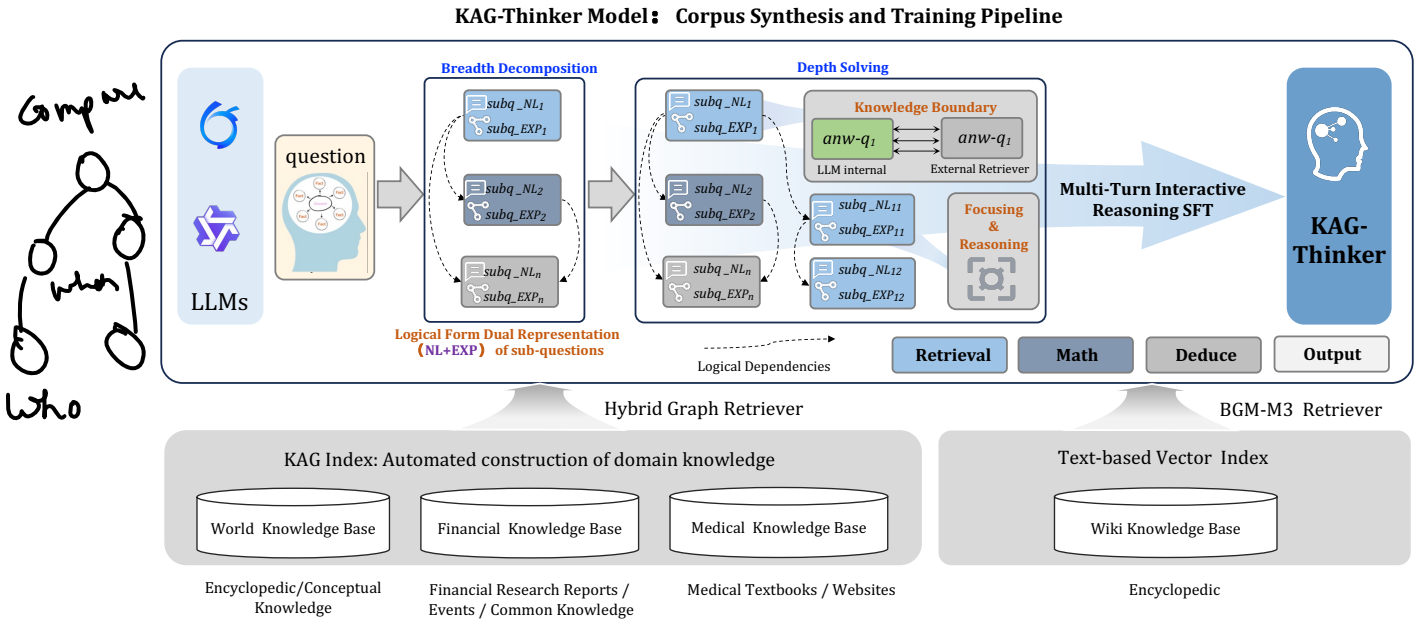

Figure 1: Overview of the KAG-Thinker reasoning model. The corpus synthesis pipeline in the figure from left to right: 1) Decompose the original $question$ into multiple sub-questions with clear logical dependencies and can be solved independently, each sub-question corresponds to a unique logical form, which consists of two components. For example, $logicalform_i$ consists of $subq\_NL_i$ expressed in natural language and $subq\_EXP_i$ expressed in logical function equivalent. The logical forms are classified into one of $Retrieval$, $Math$, $Deduce$ and $Output$ through function name. 2) Perform knowledge boundary determination and depth solving on $Retrieval$ sub-questions. During depth solving, the focusing and reasoning module reduces noise in retrieved information. 3) Finally, synthesize multi-turn interactive reasoning corpus expressed in natural language and logical function express, and obtain KAG-Thinker Model through supervised fine-tuning of Ling and Qwen's base and instruct models.

Logical Form based Reasoning. Prior RAG approaches Lewis et al. (2020); Trivedi et al. (2023); Li et al. (2025a), for both questions and sub-questions, are restricted to plain text representations, thereby preventing them from effectively utilizing high-quality structured knowledge. To address this issue, logical form proposes four operation functions tailored to frequently encountered questions (see the Appendix A). Each operation comprises two components: a natural language segment (Step) and a logical function expression segment (Action). For generic retrievers like E5 and BGE-M3, we can directly employ the content within the logical form's Step. Conversely, for structured knowledge retrievers, the Action portion of the logical form can be utilized. Crucially, logical form facilitates the propagation of dependencies among interdependent sub-problems. For example, consider *"Step1: Who is the director of Hit Parade Of 1947? Action1: Retrieval(s=s1:film['Hit Parade Of 1947'], p=p1:director, o=o1:director)"* followed by *"Step2: When did #1 die? Action2: Retrieval(s=o1, p=p2:deathtime, o=o2:deathtime)"*. The variables $\#1$ and $o1$ enable seamless transfer of text and expressions from the outcome of the first planning step. In this manner, the logical form enhances

the logic, rigor, and stability of the LLM reasoning.

Breadth Decomposition and Depth Solving. Direct retrieval often proves insufficient for complex multi-hop questions, failing to provide immediate answers. For example, *"Which film has the director who died first, Hit Parade Of 1947 or Khiladi 420?"*. To address such issues, we have devised a model based on breadth decomposition and depth solving. At a macroscopic level, the decomposition of complex questions into logical forms is generally straightforward and typically does not necessitate recourse to external knowledge. For instance, the aforementioned example can be broken down into five constituent sub-questions: *"Who is the director of Hit Parade Of 1947?", "When did #1 die?", "Who is the director of Khiladi 420?", "When did #3 die?"*, and *"Which film was directed by the director who died first according to #2 and #4?"*. The resolution of each individual sub-problem, conversely, may entail ranging from 0 to M (maximum number of searches) retrieval operations, contingent upon its inherent complexity. This in-depth approach to sub-problem solving enables each sub-problem to assimilate adequate external knowledge, thereby reaching a solvable state.

Knowledge Boundary Determination. Most prior approaches Asai et al. (2023); Gutierrez et al. (2024) initiate retrieval for every question or sub-question, even when the information resides within the LLM's inherent knowledge. However, LLMs do not necessitate external lookups for such straightforward queries. To mitigate this redundancy, we design a knowledge boundary determination module that eliminates unnecessary external retrieval operations. Prior to knowledge boundary determination, the LLM first generates answers to the sub-problems. To ensure the precision of this knowledge boundary determination, we employ two strategies: (1) prompt-based confidence assessment and (2) likelihood-based confidence assessment. Only when both conditions are satisfied is considered that the solution to the sub-problem does not require external retrieval, and the answer generated by the LLM is directly adopted.

Focusing and Reasoning. When the LLM's intrinsic knowledge proves inadequate for resolving a given sub-problem, the necessity for external information retrieval arises. However, there is a large quantity of irrelevant or low-quality external information. Following the acquisition of external data, the model undertakes a rigorous evaluation of its credibility and systematically filters out low-fidelity or irrelevant content. Subsequently, the model synthesizes this curated external information with its internal KB, progressively cultivating a comprehensive understanding of the problem through iterative reasoning cycles and knowledge refinement. This iterative process closely emulates human cognitive mechanisms for knowledge construction, acquisition, and information processing, thus critically underpinning the accuracy and reliability of the final reasoning outcomes.

Ultimately, our research yields a human-like structured iterative thinking and depth solving model offering substantial advancements, notably: (1) Facilitating more effective utilization of high-quality KBs, (2) Contributing to a more logically coherent, rigorous, and stable approach to complex problem solving, (3) Enabling autonomous decisions regarding the reliance on intrinsic model knowledge versus external information, thereby reducing unnecessary queries, and (4) Exhibiting a greater capacity to manage noise within retrieved data.

2 Related Work

Retrieval-Augmented Generation (RAG) integrates relevant, real-time information from external KBs through a retrieval mechanism, thereby enhancing its responses with contextual richness and factual grounding Borgeaud et al. (2022); Dong et al. (2023); Luo et al. (2024). Despite these advantages, a persistent challenge in RAG systems is that retrieved content can often be mixed with irrelevant or erroneous information due to the inherent limitations of current retrieval solutions. Moreover, predefined reasoning adopts structured and modular RAG pipelines that limit flexibility in evolving and open-ended tasks Li et al. (2025b).

To mitigate these challenges, researchers have introduced various components designed to enhance the robustness and effectiveness of RAG systems Gao et al. (2023); Li et al. (2024); Tan et al. (2024). A key focus in recent studies has been on enabling RAG systems to autonomously determine when to leverage retrieval mechanisms. For example, Jin et al. (2025b); Sun et al. (2025); Wang et al. (2025c) introduced a special token to explicitly trigger an external search, with the retrieval results subsequently integrated into the model to support content generation. However, Ren et al. (2025) identified limitations in LLM regarding their ability to accurately distinguish between internal and external knowledge, as well as their difficulty in reconciling conflicts between the two. To address this, they proposed Prior judgment and Poster judgments to evaluate confidence levels and assess response correctness. In parallel, another significant line of research has focused on developing adaptive strategies to optimize how retrieval is utilized, based on query complexity. Jeong et al. (2024) developed a classifier to intelligently decide between iterative, single, or no retrieval strategies depending on the complexity of the query. Building upon this idea, Islam et al. (2024) proposed a hybrid adaptive retrieval method: The model initially generates an answer without retrieval and then dynamically decides the need for retrieval based on its confidence, with the aim of balancing performance gains with inference speed. Furthermore, Tang et al. (2025) extended this work by applying a reinforcement learning-based framework to select the most appropriate retrieval strategy for a given query. While these diverse approaches have significantly advanced

the integration and adaptive use of retrieval mechanisms within RAG systems, they currently lack a comprehensive, step-by-step framework for effectively addressing complex multi-hop reasoning problems. This critical area remains an open challenge for further research.

3 Approach

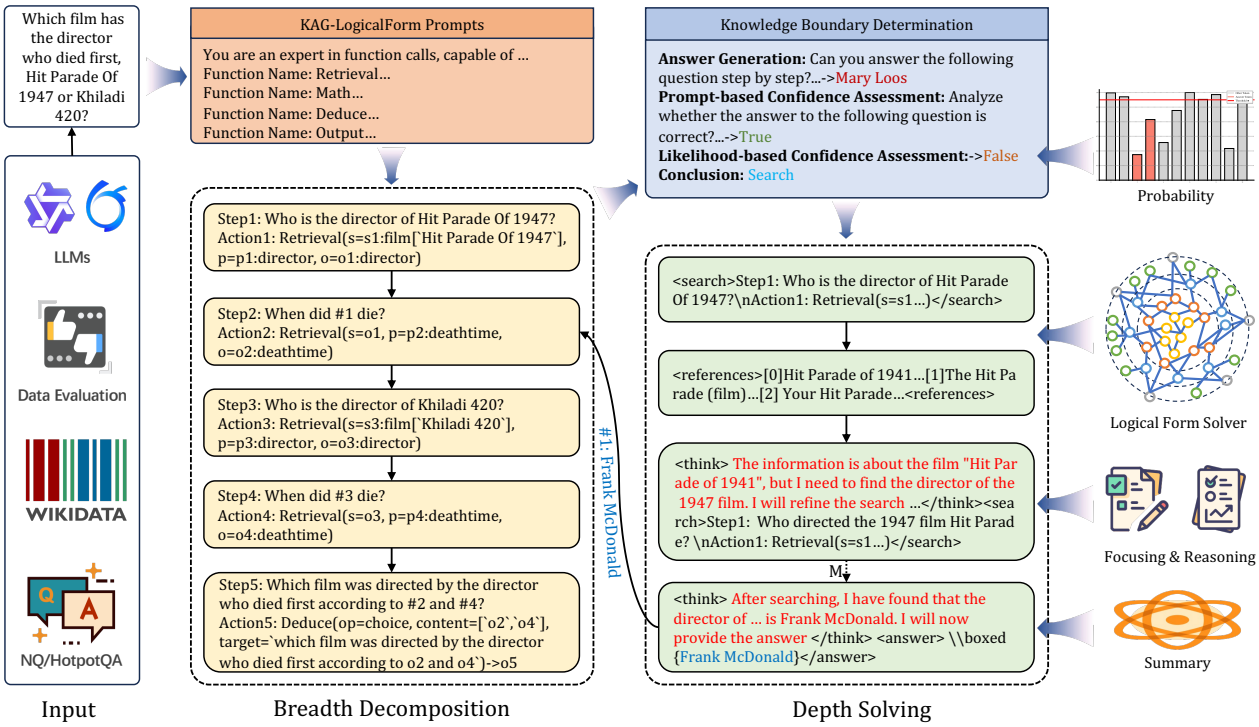

Figure 2: Overview of the problem-solving model that uses human-like reasoning process. During problem breadth decomposition, all sub-problems are obtained in a single decomposition pass, where each sub-problem is an atomic problem that can be solved independently, also known as a logical form. Herein, the terms **Step** and **Action** maintain semantic consistency, both denoting such a sub-problem. Within problem breadth decomposition, Step employs $\#n$ for answer propagation of the n-th sub-problem, while Action binds variables in logical function (e.g., o_n, s_n) for variable transmission. By determining the knowledge boundary of the *Retrieval* sub-problem, it is decided whether to utilize the base model's answer or to generate a deep retrieval. During depth solving of sub-problems, the system sequentially executes retrieval, focusing, and reasoning in iterative processes until either the sub-problem answer is obtained or the maximum solving attempt threshold is activated.

We propose an innovative complex problem solving model, as shown in Figure 2. At its core, this model uses logical form to enable both broad problem decomposition and in-depth problem solving. By integrating a knowledge boundary determination module, we minimize unnecessary external queries, balancing the LLM's internal knowledge with external information. For sub-problems that require external information, a dedicated focusing and reasoning module is employed to reduce noise in the retrieved content and extract essential information.

3.1 Breadth Decomposition and Depth Solving

Complex multi-hop problems usually need to be broken down to be solved. The key question is how to break them down effectively to best use our existing KBs. We have created logical forms to represent sub-problems. Our method decomposes problems into two parts: broad decomposition, which ensures the main problem and sub-problems stay logical and precise; and in-depth solving, which makes sure *Retrieval* sub-problem gets enough knowledge to be solved.

3.1.1 Breadth Decomposition

We decompose complex problems breadth-wise into n atomic granularity sub-problems, with our decomposition template detailed in Table 1. We define four logical form functions (Retrieval, Math, Deduce, and Output), each dedicated to handling specific tasks: retrieval-focused problems, mathematical computation and causal reasoning tasks, and results aggregation functions. Detailed specifications for these modules can be found in Appendix A. As shown

in Table 1, our breadth decomposition divides the original question into five logical forms of atomic granularity, each independently solvable. The dependency variables are propagated between logical forms by function variable $\#n$, o_n, and s_n. This approach ensures logically coherent question-solving while maintaining consistent sub-question granularity. To ensure compatibility with both natural language and Logical Form retrievers during sub-question solving, each sub-question adopts dual representations, **Step** and **Action**, that maintain semantic equivalence.

You are an expert in function calls, capable of accurately understanding function definitions and precisely decompose user queries to select appropriate functions to solve problems. The functions are as follows:
Function Name: Retrieval Description: Search for SPO information. S stands for subject, O stands for object, represented as variable_name... Function Usage: Retrieval(s=s_alias:type[‘name’], p=p_alias:edge, o=o_alias:type[name], p.prop=‘value’, s.prop=‘value’, o.prop=‘value’)
Function Name: Math Description: Perform calculations, which include set operations such as numerical calculations or sorting... Function Usage: Math(content=[‘known conditions’ or ‘o_alias/s_alias’], target=‘goal to be solved’)->math_alias
Function Name: Deduce Description: Inference refers to the process of inferring search or calculation results to answer questions... Function Usage: Deduce(op=judgement|entailment|extract|choice|multiChoice, content=[‘known conditions’ or ‘o_alias/s_alias’], target=‘goal to be solved’)->deduce_alias
Function Name: Output Description: Directly output A, B, ... as the answer, where A and B are variable names referencing previous... Function Usage: Output(A,B,...)
Please, based on the definition of the above function, decompose the user question into one or multiple logical steps, outputting the execution plan for each step along with the corresponding action. Please note: Step: Accurately point out the logical thinking process of the question, and use #1 to refer to the solution result of Step1, #2 to refer to the solution result of Step2, and so on Action: Indicate exactly the function you selected and its parameters.
Question: Which film has the director who died first, Hit Parade Of 1947 or Khiladi 420?
Output: <think>This is a comparison problem. To address the question of which film, between "Hit Parade Of 1947" and "Khiladi 420" was directed by the director who passed away first, we will undertake a chronological investigation. Our initial step involves identifying the director of "Hit Parade Of 1947". Following this, we will ascertain the director of "Khiladi 420". With both directors' identities established, we will then determine which of these individuals died earlier. This chronological analysis will enable us to connect the director's lifespan to the films they directed, ultimately revealing which film was associated with the director who died first. By meticulously comparing the timelines of the directors' lives, we ensure that our conclusion is based on accurate historical data.</think>\n\n<answer>Step1: Who is the director of Hit Parade Of 1947?\nAction1: Retrieval(s=s1:film[‘Hit Parade Of 1947’], p=p1:director, o=o1:director)\nStep2: When did #1 die?\nAction2: Retrieval(s=o1, p=p2:deathtime, o=o2:deathtime)\nStep3: Who is the director of Khiladi 420?\nAction3: Retrieval(s=s3:film[‘Khiladi 420’], p=p3:director, o=o3:director)\nStep4: When did #3 die?\nAction4: Retrieval(s=o3, p=p4:deathtime, o=o4:deathtime)\nStep5: Which film was directed by the director who died first according to #2 and #4?\nAction5: Deduce(op=choice, content=[‘o2’,‘o4’], target=‘which film was directed by the director who died first according to o2 and o4’)->o5 </answer>

Table 1: Instruction template for breadth decomposition. The template initiates with problem positioning and analysis to determine decomposition steps, ultimately generating all atomic sub-problems through single-pass decomposition. Sub-question dependencies are propagated via $\#n$ in natural language Steps and referenced through logical function variables (e.g., o_n, s_n) in Actions.

3.1.2 Depth Solving

Constrained by sub-problem representations and retriever capabilities, many sub-problems defy resolution through single-turn retrievals. We have engineered a depth-oriented resolution strategy for *Retrieval* sub-problem, as detailed in the right part of Figure 2. During depth solving, we prompt the LLM to iteratively search across multi-level and multi-dimensional contexts. The iterative process continues until one of the following conditions is met: (1) the maximum number of turns is reached, or (2) the model generates a conclusive response enclosed between the designated answer tokens <answer> and </answer>. Before conducting depth solving of the *Retrieval* sub-problem, we first perform a knowledge boundary determination on the sub-problem to assess whether the LLM can directly answer it. If the LLM can confidently answer the current sub-problem, there is no need for depth solving, and the answer can be directly obtained; otherwise, an depth solving process will be initiated. After retrieving relevant content, we conduct focusing and reasoning analysis, allowing the model to determine contextually whether to initiate the next action. If a subsequent action is required, a new Logical Form is generated; otherwise, the sub-question answer is produced. This process iterates interactively until either the maximum number of turns is reached or the sub-question answer is obtained.

3.2 Knowledge Boundary Determination

When addressing complex problems, LLMs commonly decompose them into a series of constituent sub-problems. For the resolution of these sub-problems, two principal strategies are conventionally employed: (1) direct leveraging of the model's intrinsic KB, and (2) external information retrieval to augment the solution's correctness and scope. However, each of these approaches presents distinct inherent limitations:

Limitations of Internal Knowledge Utilization. The training paradigm based on maximum likelihood estimation during the pre-training phase induces a strong generative propensity within the model's parameter space. Consequently, when confronted with ambiguous or unknown information, the model can generate erroneous or fabricated responses. This phenomenon, widely recognized as the hallucination problem, stems from the model's unwarranted epistemic overconfidence and significantly compromises the veracity of the final output.

Limitations of External Knowledge Retrieval. External knowledge retrieval is indeed capable of mitigating limitations stemming from a model's intrinsic KB. However, indiscriminate retrieval across all sub-problems engenders significant inefficiencies, manifesting as superfluous computational expenditure and redundant acquisition of information already encoded within the model's parameters.

Therefore, our work is mainly focused on reasonably determining the knowledge boundary of the model. To achieve this, we propose a **Generate First, Then Assess** strategy. Specifically, the model first attempts to directly generate answer to the sub-problem. The prompt is shown in Table 2.

Can you answer the following question step by step?
If you can, wrap your answer with <answer>\\boxed{your answer}</answer>.
If you can't, just reply that based on my internal knowledge, I can't answer this question, I need to retrieve external knowledge.
Question: Who is the author of The Pequod?

Table 2: Instruction template for solving sub-problem.

Next, we combine two confidence assessment methods: prompt-based confidence assessment and likelihood-based confidence assessment, to assess the reliability of the generated answer. The advantage of this approach is that it allows the model to first try to use the internal KB to answer the question. For a sub-problem with high enough confidence in the generated answer, this generated answer will be directly used as the final answer. Otherwise, the final answer will be obtained with the help of external knowledge.

1) Prompt-based Confidence Assessment. This confidence assessment method involves gauging confidence by asking the model sub-problems and assessing the authenticity of its generated answer. Specifically, after the model answers a sub-problem, the sub-problem and the generated answer are fed back into the model, prompting it to determine whether they match. Through this prompt-based interactive method, the model can leverage its internal knowledge to reflect and verify its generated answer step by step, ultimately generating a judgment of *True* or *False*. The details of the prompt are shown in Table 3.

Analyze whether the answer to the following question is correct?
Please think step-by-step before arriving at your conclusion.
If yes, answer True; if no, answer False.
Wrap your answer with <answer>\\boxed{True/False}</answer>.
Question: Who is the author of The Pequod?
Answer: Herman Melville

Table 3: Instruction template for confidence assessment. Where Question represents the sub-problem to be solved, and Answer represents the LLM-generated answer.

2) Likelihood-based Confidence Assessment. Confidence assessment based on likelihood is a method that evaluates the reliability of generated answers by utilizing the probabilities of output tokens. As shown in Figure 3, when the model processes a given sub-question input $\boldsymbol{x}$, it first extracts the content within \boxed{}. Then, this extracted content undergoes relocation, and the relocated result is recorded as $\boldsymbol{y} = \{y_1, y_2, \ldots, y_T\}$, where T is the length of the recorded answer sequence. During the generation process, each token y_t ($t = 1, 2, \ldots, T$) is associated with a generation probability $P(y_t \mid \boldsymbol{x}, y_{<t})$, which represents the conditional probability of generating y_t given the input and the context of previously generated tokens. The confidence of the generated answer can be evaluated based on these generation probabilities. However, directly taking the average of all token probabilities is not suitable for confidence evaluation, as certain high-frequency tokens (e.g., "the" or "of") tend to have inherently high probabilities during generation. These tokens contribute little to evaluating the model's reliability in generating critical content. To better capture areas of

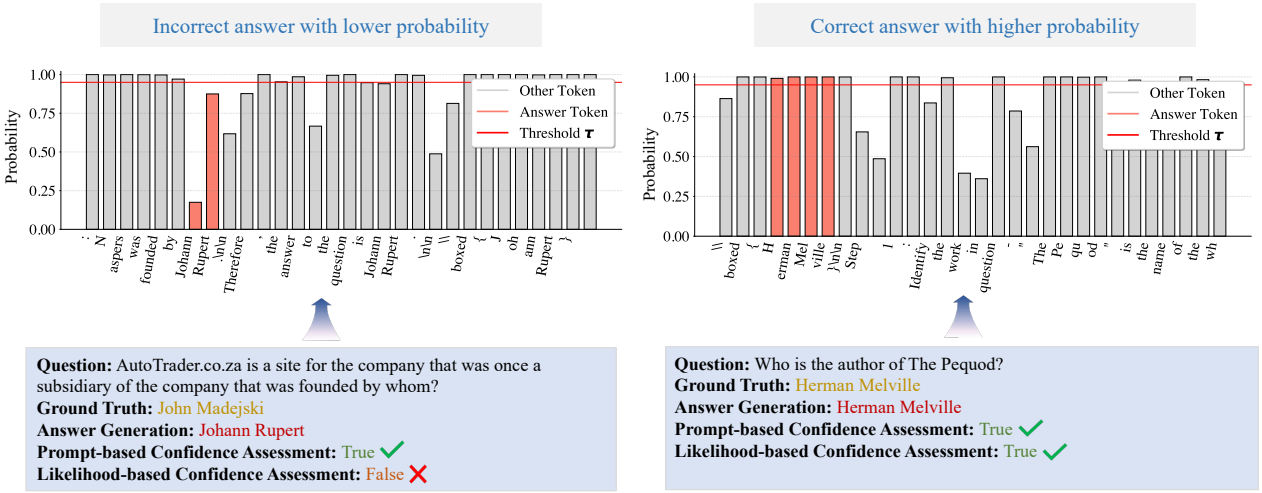

Figure 3: The probability distribution of the answer tokens. The left shows the low confidence answers, and the right shows the high confidence answers.

higher uncertainty during generation, we select the lowest generation probability among all tokens and define it as the confidence score C of the result:

$$C = \min\{p(y_1 \mid \boldsymbol{x}), p(y_2 \mid \boldsymbol{y}_{<2}, \boldsymbol{x}), \ldots, p(y_T \mid \boldsymbol{y}_{<T}, \boldsymbol{x})\}$$

According to the question and references, analyze the relationship between each reference and the question, then summarize whether these references can answer the above questions and give reasons. Wrap the answer and the corresponding reasons in <answer>Yes/No</answer> and <reason>your reason</reason>.
References: 0. "Splash (U.S. TV series)" Splash (U.S. TV series) Splash is an American competition-based reality show with celebrity diving competitions which broadcast on ABC from March 19 to May 7, 2013. It was based on the Dutch reality franchise ""Celebrity Splash!"" created by Eyeworks for the series ""Stars Jumping on Saturday"" that premiered in 2012. The show was hosted by actor Joey Lawrence and sportscaster Charissa Thompson with former Olympic divers Steve Foley from Australia and David Boudia from the U.S. as judges. Greg Louganis, also a former U.S. Olympic diver, was the diving instructor and mentor to the celebrities. The series was first 1. "Celebrity Splash!" Celebrity Splash! Celebrity Splash! is a reality television franchise created by Dutch company Eyeworks, started from their Dutch reality show ""Sterren Springen Op Zaterdag"" which premièred in 2012. The franchise involves celebrities diving into the pool. ""Splash"" has its origin and idea from German Olympic-themed variety TV show """" (""TV total Diving""), it was first aired on 16 December 2004, in the ""TV total"" show, on ProSieben and was founded by Stefan Raab and hosted by Sonya Kraus. Other hosts/reporters include Ingolf Lück (2004), Kai Pflaume (2005), Oliver Welke and (2007, 09), Steven Gätjen (2011–12) and (2011–12 reporter). It has 2. "Celebrity Splash! (Argentinian TV series)" Celebrity Splash! (Argentinian TV series) Splash! is a reality television show which teaches celebrities the art of diving. The first series is featured to start on 11 June 2013, and it will be broadcast by Telefe. Marley Wiebe will be hosting the show, with Pampita Ardohain, Maximiliano Guerra, Miguel Ángel Rodríguez and Mariana Montes as judges. ""Splash"" has its origin and idea from German TV Olympic-themed variety show (""TV Total Diving""), it was first aired on 16 December 2004, in the TV total slot, on ProSieben and was founded by Stefan Raab and hosted by Sonya Kraus. Other hosts/reporters
Question: Who is the host of Splash!
Output: </answer>Yes</answer> <reason>The host of the U.S. TV series Splash is actor Joey Lawrence, alongside sportscaster Charissa Thompson. This is explicitly confirmed in reference 0, which details the U.S. version of the show. References 1 and 2 discuss the Dutch and Argentinian iterations of the franchise, which have different hosts (e.g., Sonya Kraus, Marley Wiebe) and are unrelated to the U.S. version.</reason>

Table 4: Focusing and Reasoning Data Construction Example: By analyzing and evaluating whether the reference materials can sufficiently address the sub-question, the system determines whether to proceed to the next Action or directly generate the final answer.

Once the confidence score C is obtained, we can introduce a predefined threshold τ to further determine whether the generated answer is considered reliable. If the confidence score C satisfies: $C \geq \tau$, then the confidence of the generated answer is considered *True*; otherwise, it is considered *False*.

After obtaining the confidence results from the two assessment methods, we adopt a dual validation strategy to combine them. Specifically, the final confidence is judged as *True* only when both confidence assessment methods independently yield *True*. This indicates that the model's generated answer is reliable and can be directly utilized. Conversely, if either of the two assessment methods results in *False*, the final confidence is deemed *False*. In this case, the model's generated answer is considered potentially unreliable and requires external knowledge to obtain a more accurate answer.

3.3 Focusing and Reasoning

During knowledge boundary determination, we ascertain whether a sub-question requires retrieval. After executing retrieval, we need to assess if the current retrieval results can adequately address the sub-problem. Therefore, we have designed Focusing and Reasoning modules to fulfill this purpose. Focusing and Reasoning are primarily designed to analyze reference materials, with their data construction methodology as detailed in Table 4. Subsequently, based on the answer and reason from the Output section of Table 4, we employ LLMs to perform conditional restructuring, ultimately embedding the restructured content into the designated <think> within the depth-solving module.

3.4 Logical Form Solver

Logical Form, through its dual representation, not only ensures more rigorous logical propagation between sub-problems but also accommodates both natural language retrievers and logical form solvers. The retrieval and reasoning of sub-questions are implemented by Logical Form Solver in KAG framework, which consists of four types of logical form executors: **Retrieval Executor** is used to retrieve knowledge from an LLM's internal KB or external KB. **Math Executor** and **Deduce Executor** are utilized for Reasoning Analysis. Math is used for math or code calculations, and Deduce is an LLM causal reasoning task. **Output Executor** is used to determine and output the answer to the question. These four executors are used to connect LLMs with external executors, transforming natural language sub-problems into a structured problem-solving process. The grammatical structure of the logical form is explained in Appendix A. Table 5 shows an example of using these four executors to solve the problem of *calculating the amount of credit card fraud crime*. During the inference phase, KAG-Thinker generates equivalent representations of natural language and logical function for each logical form (such as Step1 and Action1 in table 5), and then calls the corresponding Executor based on the dependencies.

Retrieval. The logical function of Retrieval is defined as $Retrieval(s, p, o, p.contraints, p.constraints, o.constraints)$. The function variables are composed in the form of Subject-Predicate-Object (SPO) (s, p, o) together with conditional constraints. For example, *"Who was the director of the 2002 film Men in Black?"*, which can be transformed into $Retrieval$ (*s: Movie[Men in Black], p: DirectedBy, o: Person, s.ReleaseYear=2002*). The constraints can be applied to s, p, or o. The final retrieval results include the subgraph $SG(s, p, o)$ and chunks related to target variable object o.

Question: Zhang found a wallet on the street, which contained 5,000 yuan in cash and a credit card. Zhang took the cash and used the credit card to make purchases at four stores: A, B, C, and D, for 210.4 yuan, 569.2 yuan, 1035.2 yuan, 2044.5 yuan, and 1035 yuan, respectiely. After the incident, the public security authorities determined that Zhang was suspected of embezzlement and credit card fraud. What is the total amount involved in the credit card fraud charge against Zhang?

Steps	**Actions**	**Result**
1. What's the elements of the offense credit card fraud?	Retrieval(s=s1:offense[credit card fraud], p=p1:found, o=o1:elements)	o1 = "The elements of the offense of credit card fraud include the following points: 1…."
2. Which amounts are considered part of the credit card fraud amount?	Deduce(op=extract, content=[o1, Zhang found a wallet on the street…], target=Which amounts are considered part of the credit card fraud amount?)->o2	o2 = "The total amount involved in Zhang's credit card fraud is 210.4 yuan, 569.2 yuan, 1035.3 yuan, 2044.5 yuan, and 1035 yuan."
3. What's the total amount involved in Zhang's credit card fraud?	Math(content=[o1, o2], target=What's the total amount involved in Zhang's credit card fraud.)->o3	o3 = "the total amount involved in Zhang's credit card fraud is 4989.40 yuan."
4. Output #3.	Output(o3)	Amount: 4894.40 yuan.

Table 5: An example of the working principle of the Logical Form Solver in the integrated application of four executors: It search and references relevant legal provisions by **Retrieval**, extracts the amount of each illegal use of the credit card in the question by **Deduce**, calculates the amount involved in the credit card fraud case by **Math** and finally **Output** the answer.

In the recently released KAG 0.8, we have enhanced the Hybrid Graph Retriever (HGR)'s knowledge base index management capabilities, with built-in support for basic index types such as KnowledgeUnit, AtomicQuery, Outline, Summary, Chunk, Table, etc. to better support the knowledge structuring of unstructured documents. For detailed introduction, please refer to the Release Note[1].

Deduce. The logical function of Deduce is defined as $Deduce$ (`op`, `content`, `target`) $\rightarrow y_d$, where `op` specifies the reasoning method, including *extract*, *judgement*, *entailment*, and *choice*. The `content` field specifies the reasoning context, which includes the information related to the current sub-problem extracted from the user question and the calculation result of the previous logical forms passed through the intermediate variable. The `target` represents the current objective of the Deduce operation, and y_d represents the result variable of the Deduce reasoning.

Math. The logical function of Math is defined as $Math$ (`content`, `target`) $\rightarrow y_m$, where `content` and `target` are similar to Deduce, specifying the known conditions required for the calculation and the computation task, respectively. y_m represents the result of the computation.

Output. When the question can be answered using either the model's internal knowledge or the output of previous executors, the output executor will then determine the format and deliver the final answer.

4 Experiments

4.1 Experimental Settings

Benchmarks. Our experiments are conducted on 7 widely-used datasets. The dataset comprises two primary categories: (1) Single-Hop QA: NQ Kwiatkowski et al. (2019), TriviaQA Joshi et al. (2017), and PopQA Mallen et al. (2023). (2) Multi-Hop QA: HotpotQA Yang et al. (2018), 2WikiMultiHopQA Ho et al. (2020), Musique Trivedi et al. (2022), and Bamboogle Press et al. (2023). These datasets encompass diverse search and reasoning challenges, thereby enabling comprehensive evaluation of our model. Our evaluation set maintains methodological consistency with established prior works Chen et al. (2025); Jin et al. (2025b).

Evaluation Metric. While EM Yu et al. (2024) provides strict verification of answer precision; however,its binary nature is overly restrictive for open-ended scenarios where semantically equivalent answers may exist legal phrasing differences. The F1 metric therefore serves as a nuanced supplement, particularly valuable when evaluating free-form textual output.

Comparison Methods. To systematically assess KAG-Thinker's effectiveness, we establish a three-tiered evaluation model comprising the following state-of-the-art baselines: (1) **Non-Retrieval Paradigms** Naive Generation: Direct answer synthesis without external knowledge integration; Chain-of-Thought (CoT): Explicit cognitive pathway formalization through sequential reasoning traces Wei et al. (2022). (2) **Retrieval-Augmented Architectures** Naive RAG: Standard retrieval-generation pipeline without iterative optimization Lewis et al. (2020); IRCoT: Multi-cycle retrieval-reasoning coordination with dynamic feedback mechanisms Trivedi et al. (2023); Search-o1: Agent-mediated search workflow integration in reasoning processes Li et al. (2025a). (3) **Reinforcement Learning Models** R1-Gen: Pure RL-optimized generation without search engine interfacing DeepSeek-AI et al. (2025); Search-R1: Multimodal trajectory optimization combining search interactions with reasoning steps Jin et al. (2025b); ZeroSearch: Supervised LLM transformation into dual-function retrieval module (relevant/noisy document generation) Sun et al. (2025); StepSearch: PPO-based search LLM training with incremental exploration Wang et al. (2025c).

Implementation Details. To maintain consistency with previous work, we selected NQ and HotpotQA as our training datasets. Using the data construction and evaluation frameworks outlined in Appendices B and C, we obtained a total of 71K training samples. All comparison methods use Qwen2.5-7B-Instruct as the base model. We employ E5-base-v2 Wang et al. (2024) as the retriever and utilize Wikipedia data from December 2018 as the knowledge base Karpukhin et al. (2020). All corpus indexing and embedding processes are pre-processed using FlashRAG Jin et al. (2025c). Except for ReSearch, which retrieves the top 5 documents for each question, all other methods retrieve the top 3 documents. For additional implementation details, please refer to Appendix D. **Thinker** represents Thinker model, **KAG-Thinker** represents the integration of KAG framework and Thinker model.

4.2 Main Results

We evaluated our model on seven widely used benchmark datasets. Here, to maintain consistency with the retriever components of other baseline methods, we conduct plain text retrieval using the *"Step"* content from the logical form. Table 6 presents the comparisons between our model and other baselines. Compared to baselines without retrieval, the performance of our model exceeds Naive Generation and CoT by an average of 27. 1% and 34. 6%, respectively, in seven

[1]KAG v0.8 release note: https://openspg.github.io/v2/blog/recent_posts/release_notes/0.8

datasets. The primary reason for the improvement is that, within these datasets, retrieval is essential, as LLMs struggle to directly answer these questions accurately without access to relevant information. Compared to retrieval-augmented methods, our model outperforms Search-o1, IRCoT, and Naive RAG by average margins of 24.6%, 22.6%, and 14.8%, respectively. These enhancements are primarily attributed to our model's implementation of breadth decomposition and depth solving, enabling it to learn how to leverage the retriever more effectively—particularly during deep retrieval, where the generated queries align well with the retriever's capabilities. In comparison to reinforcement learning-based approaches, it can be observed that our model outperforms the previous state-of-the-art model, ReSearch, by an average of 4.1% in EM score across seven datasets. Specifically, it improves by an average of 4.5% on single-hop datasets and by an average of 3.9% on multi-hop datasets. The primary reason is that our breadth decomposition and depth solving model enables the decomposition of questions into atomic granularity, thereby reducing the complexity involved in model retrieval and answering.

	Single-Hop QA			Multi-Hop QA				Avg
	NQ†	TriviaQA*	PopQA*	HotpotQA†	2Wiki*	MuSiQue*	Bamboogle*	
Naive Generation	0.134	0.408	0.140	0.183	0.250	0.031	0.120	0.181
CoT	0.048	0.185	0.054	0.092	0.111	0.022	0.232	0.106
Search-o1	0.151	0.443	0.131	0.187	0.176	0.058	0.296	0.206
IRCoT	0.224	0.478	0.301	0.133	0.149	0.072	0.224	0.226
Naive RAG	0.349	0.585	0.392	0.299	0.235	0.058	0.208	0.304
R1-Gen	0.270	0.537	0.199	0.237	0.292	0.072	0.293	0.271
Search-R1	0.393	0.610	0.397	0.370	0.414	0.146	0.368	0.385
ZeroSearch	0.436	**0.652**	**0.488**	0.346	0.352	0.184	0.278	0.391
StepSearch	-	-	-	0.386	0.366	**0.226**	0.400	-
ReSearch	0.407	0.611	0.423	0.419	0.412	0.205	0.400	0.411
Thinker (ours)	**0.450**	0.642	0.484	**0.421**	**0.469**	0.221	**0.480**	**0.452**

Table 6: EM performance of different models on Qwen2.5-7B-Instruct. The best performance is set in bold. †/* represents in-domain / out-domain datasets. In contrast to other baselines, StepSearch and ReSearch employ the Musique dataset for training.

4.3 Thinker with KAG

We create KAG-Thinker by applying the Thinker model within the KAG framework. The multi-turn interactive reasoning framework of KAG-Thinker is depicted in Figure 4. To maintain consistency with KAG Liang et al. (2024), we continue to use its test set, with 1,000 examples each from HotpotQA, Musique, and 2Wiki (consistent with HippoRAG Gutierrez et al. (2024)). The data for our self-built corpus are entirely derived from the supporting facts of these three datasets, totaling 26,990 documents. Within the self-built corpus, and utilizing the same retriever and retrieved documents, our Thinker model still outperforms ReSearch and Search-R1 across three multi-hop datasets. For a fair comparison against

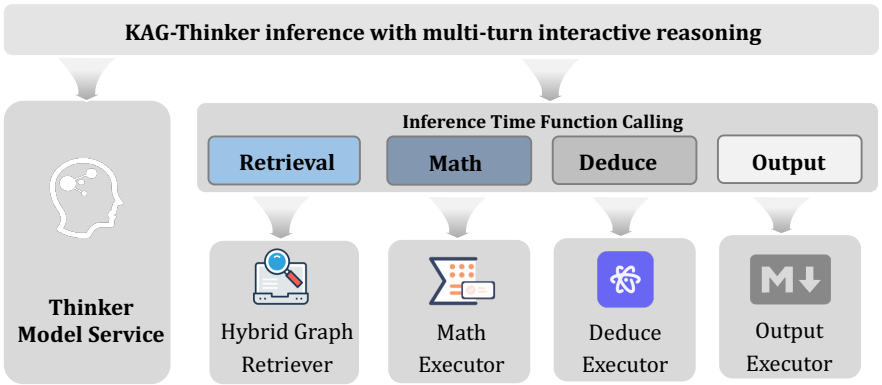

Figure 4: Overview of KAG-Thinker multi-turn interactive thinking and deep reasoning.

the baselines, the original Thinker model could only perform pure text retrieval with BGE-M3 for natural language steps in its logical form, even for operations like Deduce and Math. However, the KAG framework robustly supports these operations. Table 7 illustrates that KAG-Thinker, operating within the KAG framework, shows a marked improvement over Thinker, with average EM and F1 scores increasing by 3.0% and 3.8%, respectively. KAG-Thinker achieves these enhancements by leveraging two key features from the KAG framework: the hybrid graph retriever (HGR) and native support for Math and Deduce.

Methods	Size	Retriever	HotpotQA		MusiQue		2Wiki		Avg	
			EM	F1	EM	F1	EM	F1	EM	F1
Search-R1	7B	BGE-M3	0.545	0.676	0.307	0.403	0.517	0.589	0.456	0.556
ReSearch	7B	BGE-M3	0.538	0.671	0.322	0.423	0.555	0.633	0.472	0.576
Thinker (ours)	7B	BGE-M3	0.538	0.664	0.337	0.442	0.596	0.657	0.490	0.588
KAG-Thinker (ours)	7B	HGR	**0.568**	**0.698**	**0.350**	**0.469**	**0.644**	**0.711**	**0.520**	**0.626**

Table 7: Performance of different models within a self-built corpus. As KAG-Thinker 7B does not possess Math and Deduce execution capabilities, we utilize the executor from KAG-V0.8 72B. All retrievers retrieve the top 3 documents.

4.4 Stability Analysis

To address the inconsistency of KAG-V0.8, which often produces different solution steps on repeated attempts, we employ Thinker as the planner for KAG-V0.8. By training it on data for both breadth decomposition and depth solving, we teach the LLM to tackle problems more systematically, leading to a marked improvement in stability. For a more intuitive explanation, we introduce a stability score. For evaluation, we ask a question twice using different random seeds, thereby obtaining two distinct solution processes for each question. Then these two outputs are evaluated by LLM. If LLM determines that the two results are consistent, the question's solution steps are deemed stable; otherwise, they are considered unstable. As illustrated in Figure 5, KAG-V0.8 with Thinker outperforms KAG-V0.8 7B and KAG-V0.8 72B in terms of stability on HotpotQA, 2Wiki, and Musique. Under commonly used temperature parameters of 0.6 and 0.8, KAG-V0.8 with Thinker demonstrates an average improvement of 17.9% and 7.6% over KAG-V0.8 7B and KAG-V0.8 72B, respectively.

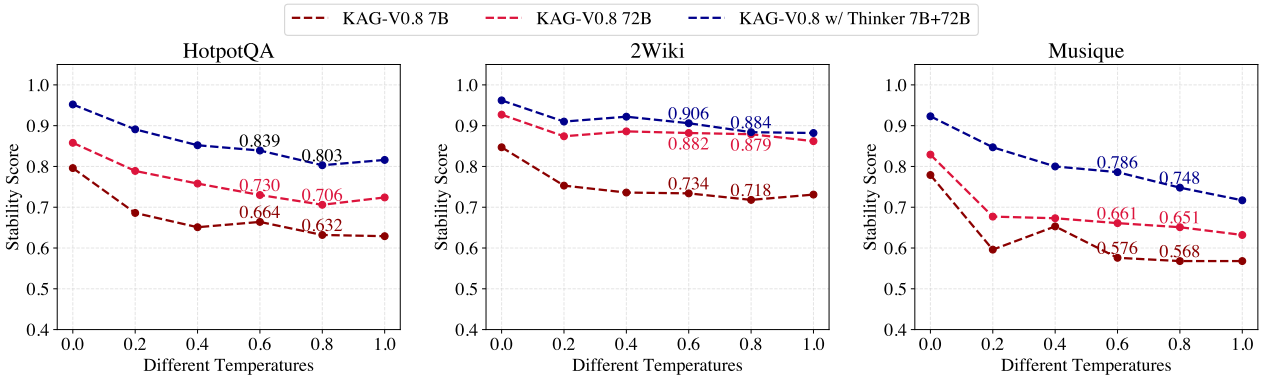

Figure 5: Stability under different temperatures.

Methods	Size	HotpotQA			MusiQue			2Wiki			AVG		
		EM	F1	LLM	EM	F1	LLM	EM	F1	LLM	EM	F1	LLM
Naive Gen	72B	0.223	0.313	0.342	0.033	0.074	0.083	0.199	0.310	0.382	0.152	0.232	0.269
Naive RAG	72B	0.566	0.704	0.762	0.248	0.357	0.384	0.448	0.512	0.573	0.421	0.524	0.573
HippoRAGV2	72B	0.557	0.694	0.807	0.289	0.404	0.452	0.542	0.618	0.684	0.463	0.572	0.648
PIKE-RAG	72B	0.558	0.686	0.787	0.383	0.498	0.565	0.630	0.720	0.810	0.524	0.635	0.721
KAG-V0.8	72B	**0.625**	**0.772**	**0.857**	**0.432**	**0.569**	**0.626**	0.692	0.784	0.847	**0.583**	**0.708**	**0.777**
KAG-V0.8	32B	0.576	0.718	0.809	0.358	0.478	0.525	**0.698**	**0.786**	**0.856**	0.544	0.661	0.730
KAG-V0.8 w/ Thinker	7B+72B	0.609	0.739	0.836	0.365	0.495	0.561	0.665	0.755	0.821	0.546	0.663	0.739

Table 8: Performance of different frameworks. The best performance is indicated in bold, and the second-best is underlined. "LLM" represents the judgment accuracy of the LLM model, which is consistent with the KAG framework. 72B refers to Qwen2.5-72B-Instruct, 32B to Qwen2.5-32B-Instruct, and 7B to the trained Thinker model.

Table 8 shows the experimental results when the planner of KAG-V0.8 is replaced with Thinker. KAG-V0.8 with Thinker demonstrates superior average performance across three datasets compared to HippoRAGV2 Gutiérrez et al. (2025) and PIKE-RAG Wang et al. (2025a). For detailed experimental settings, please refer to the KAG-V0.8 release notes. While KAG-V0.8 with Thinker significantly enhances framework stability, its average performance is lower than KAG-V0.8 72B, though it slightly surpasses KAG-V0.8 32B. This indicates that the 7B Thinker model exhibits limited problem decomposition capabilities. An analysis of bad cases reveals its struggle with complex queries. For example, "Who is the paternal grandmother of John III, Duke Of Cleves?" requires two decomposition steps (identifying John

III's mother, then her mother) which the model fails to perform. This inability is primarily attributed to two factors: inconsistent decomposition of the complex natural language question and the restricted generalization capacity of the 7B model. To address these issues, our future work will involve synthesizing problem decomposition samples from structured data, thereby enhancing LLMs' ability to decompose complex problems and improving the stability of this process.

5 Medical Field Application

In recent years, LLMs have been widely applied in the medical field. However, their limited interpretability has hindered real-world deployment, especially for complex, multifaceted problems commonly encountered in medical consultations. For instance, disease diagnosis often involves identifying candidate diseases based on symptoms and then performing differential diagnosis by reviewing the clinical manifestations of each candidate disease—a process that benefits from decomposing the problem into simpler sub-problems. Nevertheless, most existing methods attempt direct retrieval or reasoning Liu et al. (2025); Wang et al. (2025b); Tian et al. (2024); Frisoni et al. (2024); Shi et al. (2024), struggling to effectively address such intricate issues .

On the other hand, for relatively straightforward medical commonsense or symptom-related queries, such as *"What is the normal range for heart rate?"* or *"What symptoms are associated with hypoglycemia?"*, LLMs can often generate satisfactory answers. For more rigorous questions, such as those involving drug composition or dosage analysis, LLMs are prone to hallucinations, making it necessary to retrieve external knowledge. However, existing approaches Frisoni et al. (2024); Shi et al. (2024) typically do not analyze whether retrieval is truly necessary, leading to cases where retrieval introduces additional noise, particularly when the knowledge base is incomplete.

To address these issues, we design an interpretable model. By decomposing problems into logically dependent sub-problems, the problem-solving logic becomes more rigorous and interpretable. Through knowledge boundary determination, it reduces the introduction of unnecessary noisy references, thereby improving the accuracy of the model's answers.

5.1 Medical Thinker Model

To address the key points mentioned above, within the framework of KAG-Thinker, we use a breadth decomposition approach to decompose the problem into multiple sub-problems and their corresponding logical form, and we also employ knowledge boundary model to determine whether retrieval is necessary. Table 9 illustrates the reasoning process of KAG-Med-Thinker. First, within the breadth decomposition part, the content between <think> and </think> represents the thought process. Here, it is reasoned that calculating *the required fluid volume* involves considering both *the burn area and weight*. Consequently, information regarding *the percentage of lower limb (including buttock) surface area in children*, as well as *the ratio of colloidal fluids to crystalloid fluids*, should be retrieved for further decomposition of the sub-problems. Secondly, in the depth solving part, each sub-problem is addressed independently. For sub-problems related to retrieval, the system first performs its own reasoning and then evaluates whether the answer is reliable and complete. If not, it generates a retrieval directive using <search>...</search>. The Chunks retrieved for each sub-problem (with top-k=3) are integrated into the context. Since the retrieved Chunks may be lengthy, we perform knowledge compression and information targeting on the results for each sub-problem. The answers are then refined on the basis of the reference. For sub-problems requiring retrieval (e.g., Step 4 and Step 5), a deep retrieval check is employed to assess whether the Reference can resolve the sub-problem. If the Reference cannot address the sub-problem, a new retrieval directive is generated. Specifically, for Step 6, the Math directive is executed by generating Python code and obtaining computational results using a code interpreter. Finally, based on the breadth decomposition and depth solving, the complete reasoning process and the final result are generated.

We structure the samples into a multi-turn interactive reasoning format. For synthesizing data, question decomposition is performed using Qwen2.5-72B-Instruct, which offers stronger reasoning capabilities and instruction-following abilities, allowing distillation into smaller models. The target model is trained to generate answers for sub-problems and determine whether external executors are required. Since the medical Q&A often involves lengthy answers for sub-problems, we call the LLM twice to confirm whether the answers are correct and complete, which serve as labels for the knowledge boundary determination. The subsequent deep retrieval, reasoning, and answer synthesis utilize distilled synthetic data derived from the LLMs.

Question: Boy, 3 years old, weighing 16kg, sustained scald burns on both lower limbs from hot water. Physical examination: BP 85/60 mmHg, restless. Both lower limbs (including buttocks) suffered second-degree burns, with a urine output of 15ml/h. The amount of colloid fluid that should be administered in the first 24 hours is ( ). Options: A: 500ml B: 800ml C: 400ml D: 600ml E: 700ml

Breadth Decomposition:
<think> The user's question concerns a 3-year-old boy weighing 16kg...
First, we need to calculate the fluid volume based on the burn area and the child's weight...The proportion of colloid fluid to crystalloid fluid is typically 1:1 or 0.5:1, depending on the severity of the burn. Next, we need to clarify several key aspects: ...
</think>
Step 1: Fluid Requirement Formula
Action 1: Retrieval(s=s1:Medical formula['Fluid Requirement Formula'], p=p1, o=o1)
Step 2: Fluid Replacement Formula for the First 24 Hours
Action 2: Retrieval(s=s1, p=p2:time, o=o2:time['The first 24 hours'])
Step 3: The Percentage of Total Body Surface Area (TBSA) for Both Lower Limbs (Including the Buttocks)
Action 3: Retrieval(s=s2:BodyParts['Both Lower Limbs (Including the Buttocks)'], p=p3:AreaRatio, o=o3:AreaRatio)
Step 4: The Percentage of Total Body Surface Area (TBSA) for Both Lower Limbs (Including the Buttocks) in Children
Action 4: Retrieval(s=s2, p=p4:BelongTo, o=o4:Crowd['Children'])
Step 5: The Proportion of Colloid Fluid in the Total Fluid Volume for Second-Degree Burns
Action 5: Retrieval(s=s3:TreatmentPlan['Burn fluid replacement'], p=p5:Colloidal liquid ratio, o=o5:Colloidal liquid ratio, p.Degree of burn=II°)
Step 6: Based on #1, #2, #3, #4, and #5, calculate the amount of colloid fluid to be administered in the first 24 hours.
Action 6: Math(content=['o1', 'o2', 'o3', 'o4', 'o5', 'weight=16kg'], target='The amount of colloid fluid to be administered in the first 24 hours')→math6

Depth Solving:
Subanswer 1: The formula for calculating fluid replacement in burn patients is often based on the Parkland formula...
Subanswer 2: According to burn formulas, the basic principle for fluid resuscitation in pediatric burns is:
Total fluid volume for the first 24 hours = Body weight (kg) × Burned percentage (%TBSA) × 1.5 (pediatric coefficient) + baseline physiological fluid requirement.
Subanswer 3: The burn area of both lower limbs (including the buttocks) accounts for approximately 46% of the Total Body Surface Area (TBSA) in adults...
Subanswer 4: The burned area for both lower limbs (including buttocks) accounts for approximately 46% of TBSA in adults, but the percentage may differ for children and needs to be further verified. <search>Step: The Percentage of Total Body Surface Area (TBSA) for Both Lower Limbs (Including the Buttocks) in Children. Action: Retrieval(s=s2, p=p4:BelongTo, o=o4:Crowd['Children'])</search>
<Reference>...</Reference>
The area of both lower limbs (including the buttocks) for children is calculated as 46% - (12 - age)% = 37%.
Subanswer 5: Parkland Formula: 1ml of fluid per kilogram of body weight per 1% of burned area. Half of the total fluid volume is colloid fluid. However, considering the requirements for the first 24 hours and the differences in pediatric cases, I need to confirm further. <search>Step: The Proportion of Colloid Fluid in the Total Fluid Volume for Second-Degree Burns. Action: Retrieval(s=s3:TreatmentPlan['Burn fluid replacement'], p=p5:Colloidal liquid ratio, o=o5:Colloidal liquid ratio, p.Degree of burn=II°)</search>
<Reference>...</Reference>
According to the 7th Edition of the Surgery Fluid Resuscitation Protocol, the amount of colloid and electrolyte solution to be provided for pediatric patients is 2.0ml, with a ratio of 1:1 between colloid and electrolyte solution.
Subanswer 6: <Python>...</Python>
592ml

Answer:
1.Determine Burn Area:
The patient is ..., the total burn area is calculated to be 37%.
2.Calculate Colloid Volume:
According to the fluid replacement protocol ... Colloid volume = Body weight (kg) × Total burn area (%) × 1 = 16 kg × 37% × 1 = 592 ml.
Since 592 ml is approximately 600 ml, option D is selected.

Table 9: An example of the problem-solving process in KAG-Med-Thinker: The breadth decomposition module first generates the thought process, then decomposes the problem into multiple sub-problems. In the depth solving module, an adaptive retrieval equilibrium model is used for reasoning and external knowledge retrieval. Finally, the answer is generated based on the context. The red font part represents errors in the questions generated by the model, the blue font part represents adaptive generation of retrieval instructions, and the green font part represents the results executed by logical form executors.

5.2 Medical Knowledge Base

5.2.1 Medical Data Sources

We construct a Chinese medical knowledge retrieval repository containing data sourced from authoritative medical textbooks and professional medical websites. The **Medical Textbooks** originate from the Chinese materials in MedQA, specifically related to resources from the National Medical Licensing Examinations in China (e.g., the Licensed Physician Qualification Examination). These materials are highly professional and practical, making them well-suited for research and development of medical intelligent question-answering systems tailored for Chinese users. **Professional Medical Websites**. Medical data were primarily collected from five professional medical websites: Xiahe Medical Encyclopedia, DXY Doctor, Tencent Medical Encyclopedia, Kuaishou Medical Encyclopedia, and Baidu Health.

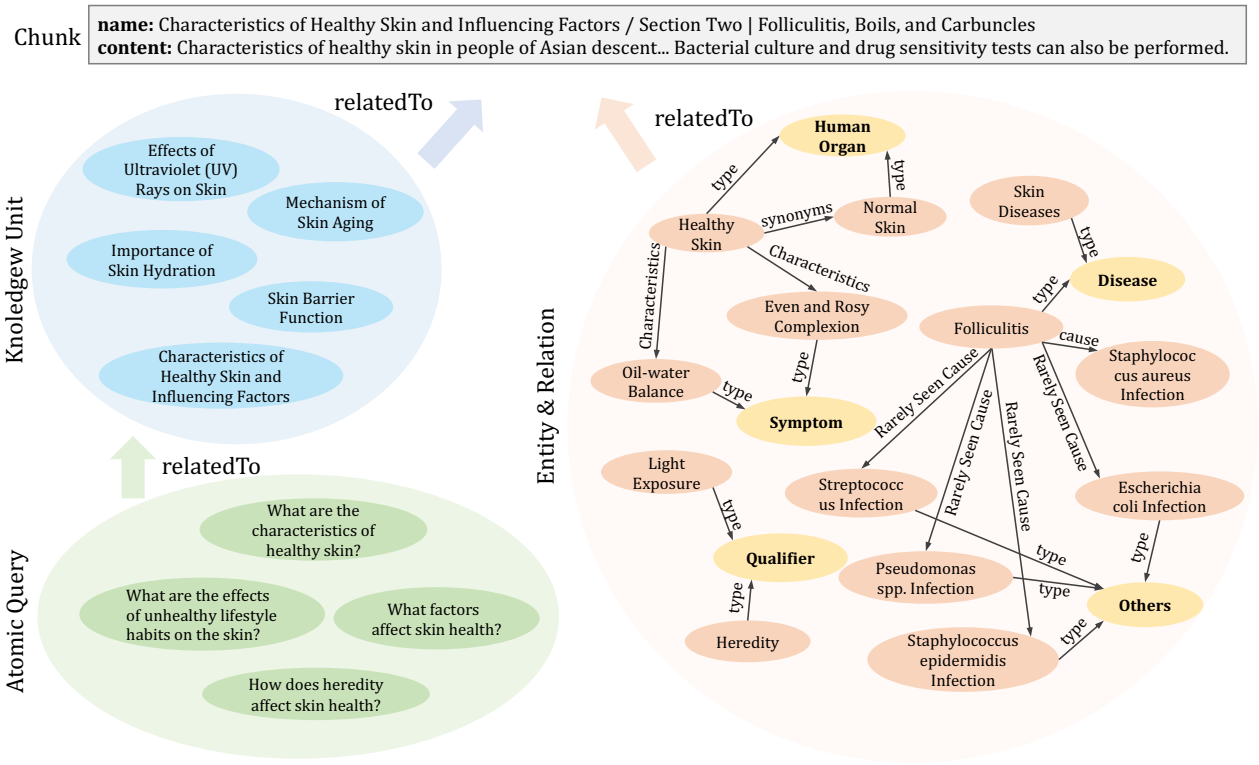

Figure 6: Medical domain knowledge base: Knowledge consists of Entities and Relations, Knowledge Units, and Atomic Query nodes. The Entities and Relations, as well as the Knowledge Units, are extracted from the original chunks. Relevant Atomic Query nodes are then generated based on the Knowledge Units. These components are interconnected and linked to the chunks.

5.2.2 Medical Knowledge Base Construction

For the extraction of information from the medical domain, we adopt schema constraint extraction due to the low error tolerance in the medical context. The use of a predefined schema restricts the extraction and generation to specific, structured key information (e.g., disease names, drug dosages, treatment plans), avoiding the introduction of irrelevant or erroneous content. We select 23 commonly used medical entity types that cover the main semantic categories involved in clinical diagnosis and treatment (e.g., diseases, symptoms, medications, diagnostic tests) and extend to general concepts related to the medical domain, public health, and system integration (e.g., disciplines, organizations, biological processes). This classification system refers to international medical terminology standards (for example, SNOMED CT, ICD-10) and ontologies of representative medical knowledges, which effectively support subsequent structured knowledge fusion, reasoning, and generation tasks. Specific medical entity types can be found in Appendix F. The detailed structured knowledge extraction process and prompt references can be found in Appendix G. Figure 6 illustrates the constructed medical knowledge base and shows the connectivity of the structured example information.

5.3 Experimental Settings and Results

5.3.1 Experimental Settings

Dataset. We selected MedQA Jin et al. (2020) as the evaluation dataset. This dataset is a collection of question-and-answer pairs sourced from professional medical examinations. All questions are multiple-choice, covering areas such as medical knowledge, disease diagnosis, pharmacology, and diagnostic procedures. Since the original test dataset is very large, we randomly sampled 300 entries from the Chinese subset to form our evaluation dataset.

Implementation Details. We conducted SFT (Supervised Fine-Tuning) training on three base models, Meta-LLAMA-3.1-8B-Instruct, Qwen2.5-14B-Instruct and DeepSeek-R1-Distill-Qwen-14B, with the following configurations: learning rate lr=5e-6, epoch=5, and using a self-constructed medical KB in Section 5.2.2, while leveraging BGE-M3 to build vector indexes.

5.3.2 Main Results

The empirical evaluation results are presented in Table 10. KAG-Med-Thinker consistently demonstrates enhanced performance across both base models, with a particularly significant improvement being observed on the Meta-LLAMA-3.1-8B-Instruct. In particular, compared to the established multi-turn planning and retrieval augmented models, IRCoT and ReAct, KAG-Med-Thinker achieves remarkable performance improvements of 11.33%, 34.78% respectively. It also outperforms the Naive RAG, by a margin of 12.46%. Among them, ReAct struggles to follow the instruction format on LLama3, resulting in poorer performance. On the DeepSeek-R1-Distill-Qwen-14B, KAG-Med-Thinker shows similar superiority, outperforming IRCoT, ReAct, and adaptive Naive RAG model by 3.95%, 4.41%, and 3.8%, respectively. The baselines based on Qwen2.5-14B-Instruct have already achieved good performance, but KAG-Med-Thinker still outperforms other reasoning and RAG models, achieving the best results. These consistent gains across different base models and comparative baselines unequivocally validate the effectiveness and robustness of KAG-Med-Thinker.

LLM	Methods	MedQA
Meta-LLAMA-3.1-8B-Instruct	Naive Generation Qwen et al. (2025)	58.33
	Naive RAG Lewis et al. (2020)	61.54
	Naive RAG+adaptive Jeong et al. (2024)	61.33
	IRCoT Trivedi et al. (2023)	62.67
	ReAct Yao et al. (2023)	39.22
	KAG-Med-Thinker	**74.00**
Qwen2.5-14B-Instruct	Naive Generation Qwen et al. (2025)	84.00
	Naive RAG Lewis et al. (2020)	81.67
	Naive RAG+adaptive Jeong et al. (2024)	85.00
	IRCoT Trivedi et al. (2023)	82.33
	ReAct Yao et al. (2023)	86.20
	KAG-Med-Thinker	**87.00**
DeepSeek-R1-Distill-Qwen-14B	Naive Generation DeepSeek-AI et al. (2025)	79.67
	Naive RAG Lewis et al. (2020)	79.00
	Naive RAG+adaptive Jeong et al. (2024)	81.48
	IRCoT Trivedi et al. (2023)	81.33
	ReAct Yao et al. (2023)	80.87
	KAG-Med-Thinker	**85.28**

Table 10: Accuracy of different models on the MedQA dataset. The best performance is set in bold.

Furthermore, it is observed that the adaptive Naive RAG consistently outperforms its exhaustive retrieval-based counterpart. In particular, it demonstrates a significant performance advantage of 3.33% on Qwen2.5-14B-Instruct and 2.48% on DeepSeek-R1-Distill-Qwen-14B. This superior performance even extends to IRCoT and ReAct, models that employ multi-turn retrieval planning. This finding shows that unconstrained or unfiltered retrieval introduces inherent limitations and noise that negatively impact overall reasoning performance. Consequently, it is essential to judiciously determine the model's intrinsic knowledge boundaries and synergistically integrate its internal knowledge with external tool invocation to fully harness their complementary strengths.

6 Conclusion

KAG-Thinker is presented as a novel, cognitively-inspired reasoning framework engineered to address the limitations of extant methodologies in solving intricate multi-hop problems demanding rigorous logical coherence and contextual consistency. The framework's core methodology emulates human cognitive processes by decomposing complex queries into logically interconnected sub-problems through a structured logical-form-based reasoning mechanism, thereby enabling incremental problem resolution while preserving structural integrity. Each logical-form task is associated with a unique logical function representation and can be solved independently. This design enables the effective decoupling of knowledge retrieval and reasoning analysis into separate subtasks. Consequently, the retrieval component can be seamlessly integrated with an automated knowledge boundary detection module, which optimizes efficiency by intelligently arbitrating between internal knowledge utilization and external information retrieval, thereby enabling the LLM to decide whether to retrieve external information based on its internal assessment. Furthermore, an integrated focusing-and-reasoning module refines retrieved content and synergizes external data with internal inferential processes, mirroring human iterative knowledge refinement. The applicability of KAG-Thinker in high-stakes domains is empirically validated via a medical Q&A system, showcasing its capacity to integrate domain-specific knowledge while maintaining logical rigor.

7 Contributors

Authors are listed **alphabetically by the first name**.

Dalong Zhang	Ling Zhong	Xinkai Du	ZhiYing Yi
Jun Xu	Mengshu Sun	YangYang Hou	Zhongpu Bo
Jun Zhou	Peilong Zhao	Yu Ao	
Lei Liang	QiWei Wang	ZhaoYang Wang	
Lin Yuan	Xiaorui Wang	Zhengke Gui	

Special acknowledgments.

Haofen Wang@Tongji University
Huajun Chen@Zhejiang University
Zhejiang University-Ant Group KG Joint Laboratory

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection, 2023. https://arxiv.org/abs/2310.11511.

Grady Booch, Francesco Fabiano, Lior Horesh, Kiran Kate, Jonathan Lenchner, Nick Linck, Andrea Loreggia, Keerthiram Murugesan, Nicholas Mattei, Francesca Rossi, and Biplav Srivastava. Thinking fast and slow in AI. In *Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Thirty-Third Conference on Innovative Applications of Artificial Intelligence, IAAI 2021, The Eleventh Symposium on Educational Advances in Artificial Intelligence, EAAI 2021, Virtual Event, February 2-9, 2021*, pages 15042–15046. AAAI Press, 2021. doi:10.1609/AAAI.V35I17.17765. https://doi.org/10.1609/aaai.v35i17.17765.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvári, Gang Niu, and Sivan Sabato, editors, *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pages 2206–2240. PMLR, 2022. https://proceedings.mlr.press/v162/borgeaud22a.html.

Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z. Pan, Wen Zhang, Huajun Chen, Fan Yang, Zenan Zhou, and Weipeng Chen. Research: Learning to reason with search for llms via reinforcement learning, 2025. https://arxiv.org/abs/2503.19470.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. https://arxiv.org/abs/2501.12948.

Guanting Dong, Rumei Li, Sirui Wang, Yupeng Zhang, Yunsen Xian, and Weiran Xu. Bridging the kb-text gap: Leveraging structured knowledge-aware pre-training for KBQA. In Ingo Frommholz, Frank Hopfgartner, Mark Lee, Michael Oakes, Mounia Lalmas, Min Zhang, and Rodrygo L. T. Santos, editors, *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM 2023, Birmingham, United Kingdom, October 21-25, 2023*, pages 3854–3859. ACM, 2023. doi:10.1145/3583780.3615150. https://doi.org/10.1145/3583780.3615150.

Giacomo Frisoni, Alessio Cocchieri, Alex Presepi, Gianluca Moro, and Zaiqiao Meng. To generate or to retrieve? on the effectiveness of artificial contexts for medical open-domain ques. *ACL*, abs/2403.01924, 2024. doi:10.48550/ARXIV.2403.01924. https://arxiv.org/pdf/2403.01924.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *CoRR*, abs/2312.10997, 2023. doi:10.48550/ARXIV.2312.10997. https://doi.org/10.48550/arXiv.2312.10997.

Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. rstar-math: Small llms can master math reasoning with self-evolved deep thinking. *CoRR*, abs/2501.04519, 2025. doi:10.48550/ARXIV.2501.04519. https://doi.org/10.48550/arXiv.2501.04519.

Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. http://papers.nips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract-Conference.html.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From rag to memory: Non-parametric continual learning for large language models, 2025. https://arxiv.org/abs/2502.14802.

Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In Donia Scott, Nuria Bel, and Chengqing Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online), December 2020. International Committee on Computational Linguistics. doi:10.18653/v1/2020.coling-main.580. https://aclanthology.org/2020.coling-main.580/.

Shayekh Bin Islam, Md Asib Rahman, K. S. M. Tozammel Hossain, Enamul Hoque, Shafiq Joty, and Md. Rizwan Parvez. Open-rag: Enhanced retrieval augmented reasoning with open-source large language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, pages 14231–14244. Association for Computational Linguistics, 2024. doi:10.18653/V1/2024.FINDINGS-EMNLP.831. https://doi.org/10.18653/v1/2024.findings-emnlp.831.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. In Kevin Duh, Helena Gómez-Adorno, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, pages 7036–7050. Association for Computational Linguistics, 2024. doi:10.18653/V1/2024.NAACL-LONG.389. https://doi.org/10.18653/v1/2024.naacl-long.389.

Bowen Jin, Jinsung Yoon, Jiawei Han, and Sercan Ö. Arik. Long-context llms meet RAG: overcoming challenges for long inputs in RAG. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025a. https://openreview.net/forum?id=oU3tpaR8fm.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning, 2025b. https://arxiv.org/abs/2503.09516.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? A large-scale open domain question answering dataset from medical exams. *CoRR*, abs/2009.13081, 2020. https://arxiv.org/abs/2009.13081.

Jiajie Jin, Yutao Zhu, Zhicheng Dou, Guanting Dong, Xinyu Yang, Chenghao Zhang, Tong Zhao, Zhao Yang, and Ji-Rong Wen. Flashrag: A modular toolkit for efficient retrieval-augmented generation research. In *Companion Proceedings of the ACM on Web Conference 2025*, WWW '25, page 737–740, New York, NY, USA, 2025c. Association for Computing Machinery. ISBN 9798400713316. doi:10.1145/3701716.3715313. https://doi.org/10.1145/3701716.3715313.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi:10.18653/v1/P17-1147. https://aclanthology.org/P17-1147/.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics. doi:10.18653/v1/2020.emnlp-main.550. https://aclanthology.org/2020.emnlp-main.550/.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi:10.1162/tacl_a_00276. https://aclanthology.org/Q19-1026/.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Xiaoxi Li, Jiajie Jin, Yujia Zhou, Yuyao Zhang, Peitian Zhang, Yutao Zhu, and Zhicheng Dou. From matching to generation: A survey on generative information retrieval. *CoRR*, abs/2404.14851, 2024. doi:10.48550/ARXIV.2404.14851. https://doi.org/10.48550/arXiv.2404.14851.

Xiaoxi Li, Guanting Dong, Jiajie Jin, Yuyao Zhang, Yujia Zhou, Yutao Zhu, Peitian Zhang, and Zhicheng Dou. Search-o1: Agentic search-enhanced large reasoning models. *CoRR*, abs/2501.05366, 2025a. doi:10.48550/ARXIV.2501.05366. https://doi.org/10.48550/arXiv.2501.05366.

Zhong-Zhi Li, Duzhen Zhang, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Pei-Jie Wang, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models. *ArXiv*, abs/2502.17419, 2025b. https://api.semanticscholar.org/CorpusID:276575321.

Lei Liang, Mengshu Sun, Zhengke Gui, Zhongshu Zhu, Zhouyu Jiang, Ling Zhong, Yuan Qu, Peilong Zhao, Zhongpu Bo, Jin Yang, Huaidong Xiong, Lin Yuan, Jun Xu, Zaoyang Wang, Zhiqiang Zhang, Wen Zhang, Huajun Chen, Wenguang Chen, and Jun Zhou. KAG: boosting llms in professional domains via knowledge augmented generation. *CoRR*, abs/2409.13731, 2024. doi:10.48550/ARXIV.2409.13731. https://doi.org/10.48550/arXiv.2409.13731.

Che Liu, Haozhe Wang, Jiazhen Pan, Zhongwei Wan, Yong Dai, Fangzhen Lin, Wenjia Bai, Daniel Rueckert, and Rossella Arcucci. Beyond distillation: Pushing the limits of medical llm reasoning with minimalist rule-based rl. *Computation and Language*, abs/2505.17952, 2025. doi:10.48550/ARXIV.2505.17952. https://arxiv.org/pdf/2505.17952.

Haoran Luo, Haihong E, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, Yifan Zhu, and Anh Tuan Luu. Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, pages 2039–2056. Association for Computational Linguistics, 2024. doi:10.18653/V1/2024.FINDINGS-ACL.122. https://doi.org/10.18653/v1/2024.findings-acl.122.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada, July 2023. Association for Computational Linguistics. doi:10.18653/v1/2023.acl-long.546. https://aclanthology.org/2023.acl-long.546/.

OpenAI. Learning to reason with llms. 2024. learning-to-reason-with-llms/.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, et al. Gpt-4 technical report, 2024. https://arxiv.org/abs/2303.08774.

Qianjun Pan, Wenkai Ji, Yuyang Ding, Junsong Li, Shilian Chen, Junyi Wang, Jie Zhou, Qin Chen, Min Zhang, Yulan Wu, and Liang He. A survey of slow thinking-based reasoning llms using reinforced learning and inference-time scaling law. *CoRR*, abs/2505.02665, 2025. doi:10.48550/ARXIV.2505.02665. https://doi.org/10.48550/arXiv.2505.02665.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models, 2023. https://arxiv.org/abs/2210.03350.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. Qwen2.5 technical report, 2025. https://arxiv.org/abs/2412.15115.

Ruiyang Ren, Yuhao Wang, Yingqi Qu, Wayne Xin Zhao, Jing Liu, Hua Wu, Ji-Rong Wen, and Haifeng Wang. Investigating the factual knowledge boundary of large language models with retrieval augmentation. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, pages 3697–3715. Association for Computational Linguistics, 2025. https://aclanthology.org/2025.coling-main.250/.

Yucheng Shi, Shaochen Xu, Tianze Yang, Zhengliang Liu, Tianming Liu, Quanzheng Li, Xiang Li, and Ninghao Liu. Mkrag: Medical knowledge retrieval augmented generation for medical question answering. *AMIA*, abs/2309.16035, 2024. doi:10.48550/ARXIV.2309.16035. https://arxiv.org/pdf/2309.16035.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Trans. Mach. Learn. Res.*, 2024, 2024. https://openreview.net/forum?id=1i6ZCvflQJ.

Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. Zerosearch: Incentivize the search capability of llms without searching, 2025. https://arxiv.org/abs/2505.04588.

Jiejun Tan, Zhicheng Dou, Yutao Zhu, Peidong Guo, Kun Fang, and Ji-Rong Wen. Small models, big insights: Leveraging slim proxy models to decide when and what to retrieve for llms. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 4420–4436. Association for Computational Linguistics, 2024. doi:10.18653/V1/2024.ACL-LONG.242. https://doi.org/10.18653/v1/2024.acl-long.242.

Xiaqiang Tang, Qiang Gao, Jian Li, Nan Du, Qi Li, and Sihong Xie. MBA-RAG: a bandit approach for adaptive retrieval-augmented generation through question complexity. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, pages 3248–3254. Association for Computational Linguistics, 2025. https://aclanthology.org/2025.coling-main.218/.

Yuanhe Tian, Ruyi Gan, Yan Song, Jiaxing Zhang, and Yongdong Zhang. Chimed-gpt: A chinese medical large language model with full training regime and better alignment to human preferences. page 7156–7173, 2024. doi:10.18653/v1/2024.acl-long.386. https://aclanthology.org/2024.acl-long.386/.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022. doi:10.1162/tacl_a_00475. https://aclanthology.org/2022.tacl-1.31/.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 10014–10037. Association for Computational Linguistics, 2023. doi:10.18653/V1/2023.ACL-LONG.557. https://doi.org/10.18653/v1/2023.acl-long.557.

Jinyu Wang, Jingjing Fu, Rui Wang, Lei Song, and Jiang Bian. Pike-rag: specialized knowledge and rationale augmented generation, 2025a. https://arxiv.org/abs/2501.11551.

Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11897–11916, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi:10.18653/v1/2024.acl-long.642. https://aclanthology.org/2024.acl-long.642/.

Xiaochen Wang, Junyu Luo, Jiaqi Wang, Yuan Zhong, Xiaokun Zhang, Yaqing Wang, Parminder Bhatia, Cao Xiao, and Fenglong Ma. Unity in diversity: Collaborative pre-training across multimodal medical sources. page 3644–3656, 2025b. doi:10.18653/v1/2024.acl-long.199. https://aclanthology.org/2024.acl-long.199/.

Ziliang Wang, Xuhui Zheng, Kang An, Cijun Ouyang, Jialu Cai, Yuhang Wang, and Yichao Wu. Stepsearch: Igniting llms search ability via step-wise proximal policy optimization, 2025c. https://arxiv.org/abs/2505.15107.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Jinyang Wu, Mingkuan Feng, Shuai Zhang, Feihu Che, Zengqi Wen, and Jianhua Tao. Beyond examples: High-level automated reasoning paradigm in in-context learning via MCTS. *CoRR*, abs/2411.18478, 2024. doi:10.48550/ARXIV.2411.18478. https://doi.org/10.48550/arXiv.2411.18478.

Ling Yang, Zhaochen Yu, Bin Cui, and Mengdi Wang. Reasonflux: Hierarchical LLM reasoning via scaling thought templates. *CoRR*, abs/2502.06772, 2025. doi:10.48550/ARXIV.2502.06772. https://doi.org/10.48550/arXiv.2502.06772.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi:10.18653/v1/D18-1259. https://aclanthology.org/D18-1259/.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. https://openreview.net/forum?id=WE_vluYUL-X.

Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. Rankrag: Unifying context ranking with retrieval-augmented generation in llms, 2024. https://arxiv.org/abs/2407.02485.

Appendix

A Logical Form

Logical Functions are defined as Table 11, with each function representing an execution action. Complex problems are decomposed by planning a combination of these expressions, enabling reasoning about intricate issues.

Function	Function Expression	Description
Retrieval	*Retrieval*(s = s_i:type[name], p = p_i:edge, o = o_i:type[name], s.prop = value, p.prop = value, o.prop = value)	Retrieve <s, p, o> triples. Conditions can be set as constraints, and s_i, p_i, o_i serve as variable names for reference in subsequent action planning. When referring to previously mentioned variable names, there is no need to regenerate entity types and names.
Deduce	*Deduce*(`op` = extract | judgement | entailment | choice, `content` = [A,B,...], `target` = [...]) $\rightarrow deduce_i$	The parameter `op` can specify different inference tasks, such as information extraction, judgment, logical reasoning, single-choice, multiple-choice, etc. The parameter `content` specifies contextual information and can be either texts or variables. The `target` specifies the objective of the inference.
Math	*Math* (`content` = [A,B,...], `target` = [...]) $\rightarrow math_i$	Numerical or statistical calculations can be performed. The `content` specifies contextual information and can be either text or variables. The `target` specifies the calculation objective.
Output	*Output* (A,B,...)	Directly output A, B, ... as the answer, where A and B are variable names referring to the results of previous retrieval, deduce, or calculation.

Table 11: Different functions of logical form.

B Data Construction Framework

RL based methods often struggle with controlling their outputs in a clear and organized way. They can't effectively customize the intermediate steps of reasoning while making decisions. Instead, these methods depend on random exploration within LLMs, leading to unpredictable reasoning paths. This unpredictability is especially problematic in important professional fields that need clear steps and the ability to review processes, such as: medical diagnostics, legal compliance, and financial risk assessment. To address these limitations, we have developed a systematic Search QA synthesis framework with customizable reasoning process specification, as depicted in Figure 7. Our data synthesis primarily covers the general and medical domains. For the general domain, a series of logical form prompts is designed (refer to Appendix A). Using Qwen2.5-72B-Instruct, we perform inference on the NQ and HotpotQA datasets to generate natural language to logical form training data. This data is then filtered for correctness using a data evaluation framework (refer to Appendix C). Subsequently, the first version of the KAG-LogicalForm model is trained. This model is then used to perform inference on the NQ and HotpotQA datasets. After filtering with the data evaluation framework and multiple rounds of iteration, the final KAG-LogicalForm training data, comprising 59K training samples, is obtained. Simultaneously, we also design a series of deep search and focusing & reasoning prompts (refer to Sec 3.3). We perform inference on single-hop questions within the NQ and HotpotQA datasets using Qwen2.5-72B-Instruct, which leads to a sub-question deep search model. After multiple rounds of iteration, the final KAG-DeepSearch training data, comprising 18K training samples, is obtained. Then, based on the KAG-LogicalForm and KAG-DeepSearch models, we obtain the breadth decomposition and deep solving process for each question. Finally, we perform a knowledge boundary determination for each sub-question. For sub-questions that LLMs can confidently answer, we omit their deep solving process to reduce unnecessary retrieval by the model. After multiple rounds of iteration and validation by the data evaluation framework, we obtain the final KAG-Thinker training data, comprising 71K training samples. The data construction method in the medical domain is the same as that in the general domain. The training data constructed for KAG-LogicalForm, KAG-DeepSearch, and KAG-Med-Thinker amount to 14K, 1K, and 20K entries, respectively.

C Data Evaluation Framework

The Data Evaluation Framework (DEF) constitutes a systematic methodology for optimal dataset curation through multi-dimensional quality assessment. In contemporary machine learning paradigms, empirical evidence consistently

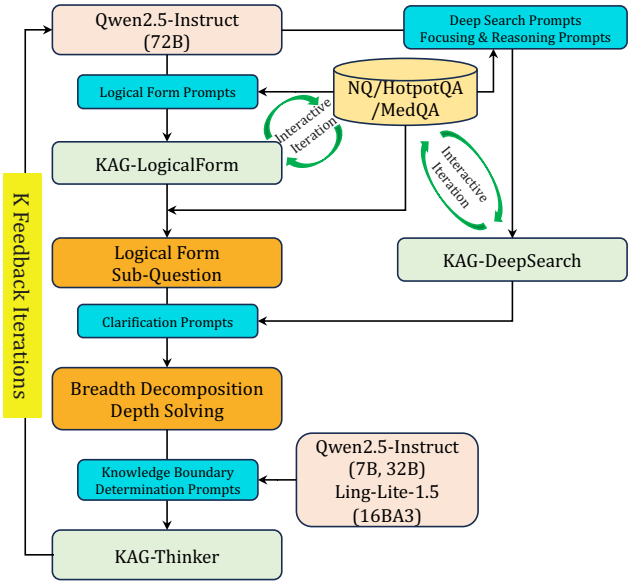

Figure 7: Overview of the data construction framework.

demonstrates that data quality exerts a more critical influence on model performance than dataset scale-a particularly salient consideration given the pervasive challenge of quality inconsistencies in publicly available corpora. Motivated by this critical need, we have engineered an evaluative architecture capable of granular quality diagnostics at the query-answer pair level. the DEF operates through six hierarchically organized assessment dimensions: security, accuracy, relevance, logic, fluency, and emotion.

Security. A security assessment identifies issues within data to ensure its safety and integrity. It checks if model outputs align with social norms, laws, and ethical standards. Key areas of focus include: (1) Content: Checking for sensitive, discriminatory, or inappropriate material to prevent harm. (2) Privacy & Security: Ensuring compliance with privacy laws and data security rules, such as preventing unauthorized access. (3) Risk: Identifying and removing statements that could cause controversy or significant risks, which helps maintain trust and reputation. (4)Consequences: Evaluating if actions based on the data might lead to negative outcomes. In essence, this systematic check ensures data is accurate, legal, and ethical. It finds and resolves potential threats, fostering trust and security.

Accuracy. Accuracy verification is a key process that identifies errors and inconsistencies in data and model outputs. It ensures model responses accurately address user questions and are factually sound. Key aspects of this evaluation include: (1) Factual Error Detection: We rigorously check for any incorrect information, such as wrong dates, data, or logical flaws, to ensure reliability. (2) User Intent Understanding: We verify that the model correctly interprets the user's question and its context, preventing irrelevant or misleading answers. (3) Common Sense & Domain Alignment: Outputs are checked against established knowledge and common sense principles for coherence and practical value. (4) Illusion Detection: We look for answers that might seem correct but could actually mislead the user. (5) Avoiding Over-Reasoning: We ensure the model doesn't provide excessive details or go beyond the scope of the question, keeping responses focused. (6) Rejection Criteria: We define when an answer must be rejected due to irrelevance, inaccuracy, or confusion. (7) Process Review: We examine the entire process of generating an answer for any factual errors introduced at any stage. Crucially, judgments about factual correctness are based on the common sense embedded in the question itself. While external information can be helpful, the primary focus is on the question's inherent context and the user's expectations.

Relevance. Relevance assessment evaluates whether model outputs are on-topic and appropriate for the given context. This process primarily checks how well the model's responses align with the input questions or cues. Key aspects of this evaluation include: (1) Topic Deviation: We identify if the output strays from the main topic or includes information not directly relevant to the user's query. This ensures the information stays focused. (2) Core Issue Identification: We assess the model's ability to pinpoint the main point of a question and provide specific answers that directly address it. (3) Clarity and Conciseness: We ensure responses are not vague, too general, or overly detailed, which can obscure clarity. The goal is to provide concise, useful information without unnecessary embellishment. In short, a relevance check is a systematic way to ensure model outputs are coherent and contextually appropriate. By identifying and fixing irrelevant content, we guarantee the information is not only accurate but also directly pertinent to the user's specific needs. This significantly improves the system's effectiveness and user satisfaction.

Logic. Logical integrity assessment ensures that data outputs are internally consistent and free from errors. It verifies that model responses are rational and do not contain contradictions. Key areas of focus include: (1) Logical Flow: Checking that timelines, sequences, and cause-and-effect relationships within the output are sound and consistent. (2) Reasoning Soundness: Identifying any gaps in logical reasoning, flawed arguments, or incorrect deductions that undermine the conclusions. (3) Real-World Alignment: Verifying that outputs align with established facts, common sense, and scientific principles governing real-world phenomena. Ultimately, this meticulous process identifies and corrects logical flaws, thereby strengthening the reliability of the information and fostering confidence in data-driven decisions.

Fluency. This assessment evaluates the language quality of model-generated text, focusing on its naturalness, coherence, and readability. Key aspects examined include: (1) Structural Clarity and Grammar: We verify clear sentence structures and grammatical correctness to ensure easy comprehension. (2) Conciseness: We aim to eliminate repetition, wordiness, and overly complex phrasing for clearer and more direct communication. (3) Idiomatic Usage and Style: We ensure the language sounds natural, adhering to human-like idioms and appropriate stylistic conventions. In essence, this assessment maintains high standards for linguistic excellence in model outputs. By systematically identifying and correcting fluency issues, we enhance overall text quality and readability, fostering effective communication and building trust.

Emotion. This assessment checks the emotional and stylistic tone of model-generated outputs. Its main goal is to ensure the tone is appropriate for the given context and user. Key evaluation points include: (1) Matching User Expectations: We ensure the output's tone (e.g., formal, friendly, humorous) aligns with what the user anticipates or what the situation demands. (2) Avoiding Inappropriate Expressions: We prevent the model from using blunt, indifferent, or offensive tones that could alienate or confuse users. (3) Contextual Adaptability: We verify the model can adjust its tone based on different contexts and various user preferences. In short, this assessment ensures the model communicates with emotional intelligence and the right tone. By identifying and correcting tonal issues, it fosters more meaningful interactions, improves user engagement, and builds trust in the communication.

To validate the performance of our data evaluation framework, we select positive and negative samples from the medical and legal domains for human evaluation. The evaluation metrics for positive samples are systematically presented in Table 12. Our DEF demonstrates robust performance across individual assessment dimensions, achieving accuracy rates exceeding 90% in all criteria. Domain-specific composite evaluations yield 88.8% accuracy in medical and 93.8% in legal scenarios. Crucially, the framework exhibits controlled false positive rates during positive sample validation, indicating effective preservation of valid instances. The evaluation metrics for negative samples are comprehensively detailed in Table 13. Our DEF achieves classification accuracy surpassing 80% across all individual assessment criteria, with domain-specific composite accuracy reaching 88.6% in medical and 85.2% in legal. Through continuous optimization of the DEF, the quality of our data has been steadily improving.

	Medical			Legal		
	Good Case	Bad Case	Acc	Good Case	Bad Case	Acc
Security	72	0	1.000	96	0	1.000
Accuracy	66	6	0.917	91	5	0.948
Relevance	70	2	0.972	94	2	0.979
Logic	70	2	0.972	96	0	1.000
Fluency	71	1	0.986	96	0	1.000
Emotion	70	2	0.972	95	1	0.990
All	64	8	0.888	90	6	0.938

Table 12: The data evaluation framework's performance in positive cases in medical and legal fields.

	Medical			Legal		
	Good Case	Bad Case	Acc	Good Case	Bad Case	Acc
Security	24	5	0.828	36	9	0.800
Accuracy	27	2	0.931	58	8	0.879
Relevance	24	6	0.800	42	8	0.840
Logic	25	6	0.806	43	7	0.860
Fluency	24	5	0.828	50	4	0.926
Emotion	31	4	0.886	46	8	0.852

Table 13: The data evaluation framework's performance in negative cases in medical and legal fields.

D Implementation Details

For training, we combine the NQ and HotpotQA training sets to create a unified dataset for KAGThinker. Training is performed on 8×2 Nvidia H100 GPUs, utilizing full parameter optimization and gradient checkpointing. Key parameter settings are detailed in Table 14.

Parameter	Value	Parameter	Value
Learning Rate	5e-06	Weight Decay	0.0
Training Batch Size	64	Warmup Ratio	0.06
Training Epochs	5	Gradient Clip	1.0
Max Sequence Length	16384	Seed	1024
Optimizer	Adamw	LR Scheduler	linear

Table 14: Some important parameters for model training.

E Training Sample

To more intuitively illustrate our training process, we provide a detailed example below.

```
[{
      "role": "system",
      "content": "As you answer each question, you must provide a thought process and insert it between < think > and
      ↪  </think>."
}, {
      "role": "user",
      "content": "You are an expert in function calls, capable of accurately understanding function definitions and precisely
      ↪  decompose user queries to select...Question:\nwho does the honey badger play for in the nfl\nOutput:\n"
}, {
      "role": "assistant",
      "content": "<think>This is a sports inquiry problem. Our investigation ...</think>\n\n<answer>Step1: who does the
      ↪  honey badger play for in the nfl\nAction1: Retrieval(s=s1:player[`Honey Badger`], p=p1:playsFor,
      ↪  o=o1:team)\nStep2: Output #1\nAction2: Output(o1)</answer>"
}, {
      "role": "user",
      "content": "Can you answer the following questions step by step? If you can, wrap your answer with
      ↪  <answer>\\boxed{your answer}</answer>. If you can't, just reply that based on my internal knowledge, I can't
      ↪  answer this question, I need to retrieve external knowledge. \nQuestion: \nStep1: who does the honey badger play
      ↪  for in the nfl\nAction1: Retrieval(s=s1:player[`Honey Badger`], p=p1:playsFor, o=o1:team)"
}, {
      "role": "assistant",
      "content": "<think>based on my internal knowledge, I can't answer this question, I need to retrieve external
      ↪  knowledge.</think>\n<search>Step1: who does the honey badger play for in the nfl\nAction1:
      ↪  Retrieval(s=s1:player[`Honey Badger`], p=p1:playsFor, o=o1:team)</search>"
}, {
      "role": "user",
      "content": "<references>[0]\"Tyrann Mathieu\"\nTyrann Mathieu Tyrann Devine Mathieu (; born May 13, 1992) is an
      ↪  American football safety for the Houston Texans of the National Football League (NFL)...</references>"
}, {
      "role": "assistant",
      "content": "<think> Based on the search results, the nickname \"Honey Badger\" is associated with Tyrann Mathieu,
      ↪  an American football player. I will now search for the team he plays for.</think>\n<search>Step1:  Who does
      ↪  Tyrann Mathieu play for in the NFL? \nAction1:Retrieval(s=s1:human[`Tyrann Mathieu`], p=p1:playsFor,
      ↪  o=o1:sportsTeam)</search>"
}, {
      "role": "user",
      "content": "<references>[0]\"Tyrann Mathieu\"\nthe game on a Phil Dawson field goal. On November 26, Mathieu
      ↪  recorded his second Interception of the season in a 27-24 overtime win against the Jacksonville
      ↪  Jaguars...</references>"
}, {
      "role": "assistant",
      "content": "<think> The search results indicate that Tyrann Mathieu is currently playing for the Houston Texans in
      ↪  the NFL.</think>\n<answer> The Honey Badger plays for \\[ \\boxed{Houston Texans} \\] in the NFL.
      ↪  </answer>."
}, {
```

```
    "role": "user",
    "content": "Answer the last question based on question, the first n sub-questions and their answers (indicated by #n),
    ↪  and wrap the result with <answer>\\boxed{your answer}</answer>. \nquestion:\nwho does the honey badger
    ↪  play for in the nfl\nsub-questions:\nwho does the honey badger play for in the nfl\n#1: Houston
    ↪  Texans\n\nlast-question:\nOutput #1\nanswer:\n"
}, {
    "role": "assistant",
    "content": "<think>I think I can answer this question now</think>\n\n<answer>\\boxed{Houston
    ↪  Texans}</answer>."
}]
```

F Med Entities

In the medical field, fully open-ended entity relationship extraction can result in a large amount of useless information, leading to unnecessary resource consumption and providing no benefit to retrieval. Therefore, we imposed constraints on the types of entities to be extracted. Based on the application scenarios, we identified the following 23 types of entities and only extracted relationships among these entities, with no constraints on the types of relationships. For details, see Table 15.

Entities	Describe
Disease	Names of various diseases, such as "diabetes", "hypertension", etc
Symptom	Subjective feelings or clinical manifestations of patients, such as "fever" and "headache"
SurgicalProcedure	Various surgical procedures and procedures, such as "cholecystectomy"
Drug	Chemicals used for treating, preventing, or diagnosing diseases, such as aspirin
Vaccine	Biological products for immunization, such as "hepatitis B vaccine"
Examination	Clinical physical examination items, such as "electrocardiogram examination"
LaboratoryTest	Laboratory testing items, such as "blood routine" and "urine protein testing"
Population	Specific population descriptions, such as "elderly population" and "pregnant women"
MedicalDiscipline	Medical branch disciplines, such as "Internal Medicine" and "Surgery"
Department	Diagnosis and treatment departments in medical institutions, such as "Cardiovascular Medicine"
HumanOrgan	Organs in human anatomy, such as the liver and heart
MedicalDevice	Medical equipment for clinical use, such as "ventilators" and "electrocardiogram monitors"
Organization	Medical related units or organizations, such as "hospitals" and "disease control centers"
Location	Geographical locations related to medical behavior, such as "outpatient department" and "ward"
Gene	Genetic information related to diseases or phenotypes, such as "BRCA1"
Substance	Chemical or biological substances involved in medicine, such as insulin and glucose
Qualifier	Restrictive vocabulary that modifies other entities, such as "chronic" and "acute"
Biology	The types of organisms involved, such as "Escherichia coli" and "influenza virus"
Specimen	Collect biological samples for testing, such as "blood" and "urine"
BiologicalProcess	Processes involving life activities, such as "cell apoptosis" and "inflammatory response"
PhysicalEnergy	Physical energy forms related to medicine, such as "X-rays" and "ultrasound"
Event	Specific events that occur during the medical process, such as' postoperative complications'
PhysicalEntity	Perceived and measurable physical objects, such as "blood pressure monitors" and "syringes"
ObservationTarget	Objects being observed or monitored, such as "blood glucose levels" and "temperature changes"

Table 15: Entity type constraints in the medical field

G Medical Knowledge Base Extraction

Data Processing Strategy. In this study, a hybrid semantic structural segmentation strategy is used to process the MedQA textbook data. After converting the data to Markdown format, it was split into fine-grained chunks based on chapter logic and a length limit of 1000 characters to ensure semantic integrity and preserve chapter path information. For medical website contents, we used directly structured paragraphs as independent chunks to avoid semantic fragmentation and improve retrieval efficiency.

Knowledge Extraction Pipeline. Building upon the Qwen2.5-72B-Instruct model, we developed a comprehensive knowledge extraction pipeline designed to extract medical knowledge across multiple levels, including knowledge units, atomic queries, entities, and relations. This pipeline transforms unstructured text into a unified, structured knowledge

Prompt for Entity Extraction:
You are a highly efficient medical entity extraction system. Your task is to extract all possible entities from a given piece of medical text and generate structured data according to a unified specification. We will provide you with a piece of medical text, and based on the provided "medical type list," you need to deeply analyze the content of the text to identify and extract all relevant entities. Each entity should include fields such as name, type, domain ontology, explanation, standard name, and synonyms.
Output format requirements:
Please respond in the format of a JSON list, where each entity is represented as a dictionary in the following structure:
{ "name": "The complete term or proper noun as it appears in the original text", "Type": "The type of the entity selected from the type list", "Domain ontology": "The hierarchical classification chain of the medical domain to which the entity belongs", "Explain": "A descriptive explanation of the entity based on your knowledge", "Sandard Name": "The standard name of the entity (if different)", "Synonym": ["All known synonyms of the entity"] }
Medical Entity Types:
<EntityTypes>...</EntityTypes>
Examples:
<Examples>...</Examples>
Instructions for Use:
- Input a piece of medical text.
- The output should be in JSON list format, with each entry corresponding to one entity.
- Supports the extraction of multiple main entities simultaneously.
- All entities must be derived from the provided text content.
- The extracted entities can be used for subsequent applications such as knowledge graph construction, semantic search, etc.
Input: <Chunk>...</Chunk>

Prompt for Relation Extraction:
You are an efficient medical relationship extraction system. Your task is to analyze the input text based on a given list of medical entities and extract all possible relationships that exist between these entities. The output should be in the form of a list of quadruples.
Each quadruple should include:
The subject entity. A predicate representing the relationship between the two entities. The object entity. Specific constraints (e.g., time, location, etc.). If no relationships are found, there is no need to list any output.
Output format requirements:
Respond using a JSON list format, where the entire response is a list, and each quadruple is also a list, with the format as follows:
[["Subject Entity", "Predicate", "Object Entity", "Specific Constraint"], ...]
Extraction Requirements:
1. Subject and Object Entities: Must only include entity names from the given entity list. Descriptive modifiers can be added to make the description more precise (e.g., "Insulin for intravenous injection").
2. Predicate:
- Should be meaningful terms or phrases that clearly express the relationship between the entities.
- Must have directionality, favoring concise expressions (e.g., "treats > is used to treat").
- Avoid using prepositions such as "is," "by," "in," "and," or other terms without actual semantic meaning.
- Single-word predicates should be avoided.
3. Specific Constraints:
- These are specific conditions under which the relationship between entities holds, such as time, dosage, frequency, route of administration, etc.
- Multiple constraints should be separated by commas.
- If no constraints exist, use an empty string "".
4. Source Restriction: Extracted quadruples must strictly originate from the content of the original text and not include inferred or extended relationships.
5. Formatting: Each quadruple must contain exactly four elements, separated by a comma ,.
6. Deduplication: Avoid duplicate or highly similar quadruples in the output.
7. Medical Terminology Consistency: Ensure that terminology aligns with medical standards, such as "oral administration," "intravenous infusion," or "subcutaneous injection."
<Examples>...</Examples>
Notice:
- All entity names must exactly match those provided in the entity list, with qualifiers added if necessary.
- Relationships must be derived directly from the text and not inferred based on common sense reasoning.
- If an entity appears multiple times in the text but the relationship is the same, retain only one instance.
- Specific details such as time, dosage, frequency, and route should be included as "specific constraints" in the fourth item.
Input: <Entities>...</Entities> <Chunk>...</Chunk>

Table 16: Prompt for entity and relationship extraction.

Prompt for Knowledge Point Extraction:
You are a Medical Document Analysis and Knowledge Point Extraction Assistant. Please extract knowledge points from the input document fragment (chunk) that provide a substantive description and introduction of the document's theme-related background knowledge, involved individuals, time, and location, or discuss, elaborate, or analyze relevant information in the text.
Ensure that the extracted knowledge points are strictly related to the document's theme and represent core content directly introduced, analyzed, or reasoned by the document's author in the fragment. If the knowledge point's subject cannot be clearly identified but the core entities of the document are provided in the input, you may refer to those entities.
Extraction Requirements
1. Identify the core discussion object in the fragment and extract content that contributes creatively to the theme of the document.
2. If the fragment contains a significant amount of referenced or mentioned content, only extract key knowledge points that involve further analysis, expansion, reasoning, or application related to those references. Do not include auxiliary information or background content.
3. The name of the knowledge point must be described in concise and precise language, fully reflecting the core theme of the knowledge point.
4. Each knowledge point should be a complete, coherent narrative or discussion. Knowledge points within a single chunk may include multiple items:
- When discussing the same theme or subject from multiple perspectives in parallel, split them into separate knowledge points.
- If further splitting compromises the causality, reasoning, or completeness of the discussion, treat the entire fragment as a single knowledge point instead of further fragmentation.
- Ensure the content description of the knowledge point is structured and avoids redundant or repetitive explanations.
Types of Medical Knowledge:
- Declarative Knowledge: Explanations of concepts, terms, theorems, clauses, etc., in the medical domain, answering "What is" questions (e.g., disease definitions, surgical explanations, mechanisms of drug action, examination items).
- Case-Based Knowledge: Conveying and interpreting abstract medical theories through specific clinical contexts, such as a patient diagnosed with diabetes and receiving insulin therapy.
- Factual Knowledge: Description of the state, attributes, characteristics, or background information of medical phenomena (e.g., diagnostic data, health status statistics, drug usage data).
- Procedural Knowledge: Instructions on how to perform specific medical tasks or operations (e.g., examination procedures, testing methods, surgical steps, rehabilitation training).
- Inferential Knowledge: Logic-based analysis, induction, deduction, or reasoning to arrive at medical judgments (e.g., supporting diagnostic conclusions or treatment plans).
Output Format:
{ "Knowledge Point 1 Name": { "Content": "A description or summarized text fragment extracted from the chunk that unambiguously explains the knowledge point content and subject, including key information.", "Knowledge Type": "Declarative Knowledge/Case-Based Knowledge/Factual Knowledge/Procedural Knowledge/Inferential Knowledge", "Structured Content": "Structured content associated with the knowledge point, such as charts, formulas, code, rules, first-order logic representation, equation systems, data structure definitions, etc.", "Domain Ontology": "A hierarchical classification system of the knowledge point's medical or professional domain, consisting of broader-to-narrower category terms. The current entity must not be directly placed into the domain ontology.", "Core Entities": "Keywords representing the theme or characteristics of the knowledge unit. Each term must be semantically complete and reflect the knowledge unit's uniqueness in searches. Each core entity's valid types must correspond to the 'type list' provided.", "Related Questions": "A set of questions, fully answerable based on the knowledge point's subject and content as described in the original text.", "Extended Knowledge Points": "A list of additional 'most relevant' knowledge point names representing extensions of the current knowledge point. These are closely tied to the current knowledge point, follow it in a logical sequence, and are not explicitly mentioned in the original text." },... }
Medical Entity Types
<EntityTypes>...</EntityTypes>
Input: <Chunk>...</Chunk>

Table 17: Prompt for Knowledge Point Extraction.

base. The prompt for entity-relation extraction, as shown in Table 16, first extracts entities based on the constraints of the specified entity types, and then extracts the relationships between these entities. For knowledge points extraction, as shown in Table 17, knowledge points are extracted according to the prompted medical knowledge types and entity types. Each knowledge point includes core entities, related questions, and other information, thereby building a knowledge graph that comprises entities, knowledge points, related questions, and chunks.